

Abdul Wahid

EC-Council CND (Certified Network Defender)

IoT Security – MQTT Communication, Traffic Analysis & TLS/PKI Implementation

Executive Summary

This module focuses on the implementation, configuration, and security analysis of an IoT communication environment using an MQTT-based architecture. The objective of this lab was to establish communication between virtual IoT devices and an MQTT Broker while analyzing the security challenges associated with unencrypted IoT data transmission.

The lab environment was developed using two virtual machines: a **Web Server**, which hosted the Bevywise MQTT Broker (CrystalMQ), and an **Admin Machine-1**, which hosted the IoT Simulator used to create and manage virtual IoT devices. Network communication between these systems was configured using VMware virtual networking to ensure proper connectivity between the broker and client devices.

Initially, an MQTT communication channel was established without encryption to observe the transmission of IoT messages. Wireshark was used to capture network traffic and demonstrate that MQTT messages were transferred in plaintext, allowing sensitive information such as topics and payload data to be viewed during packet analysis.

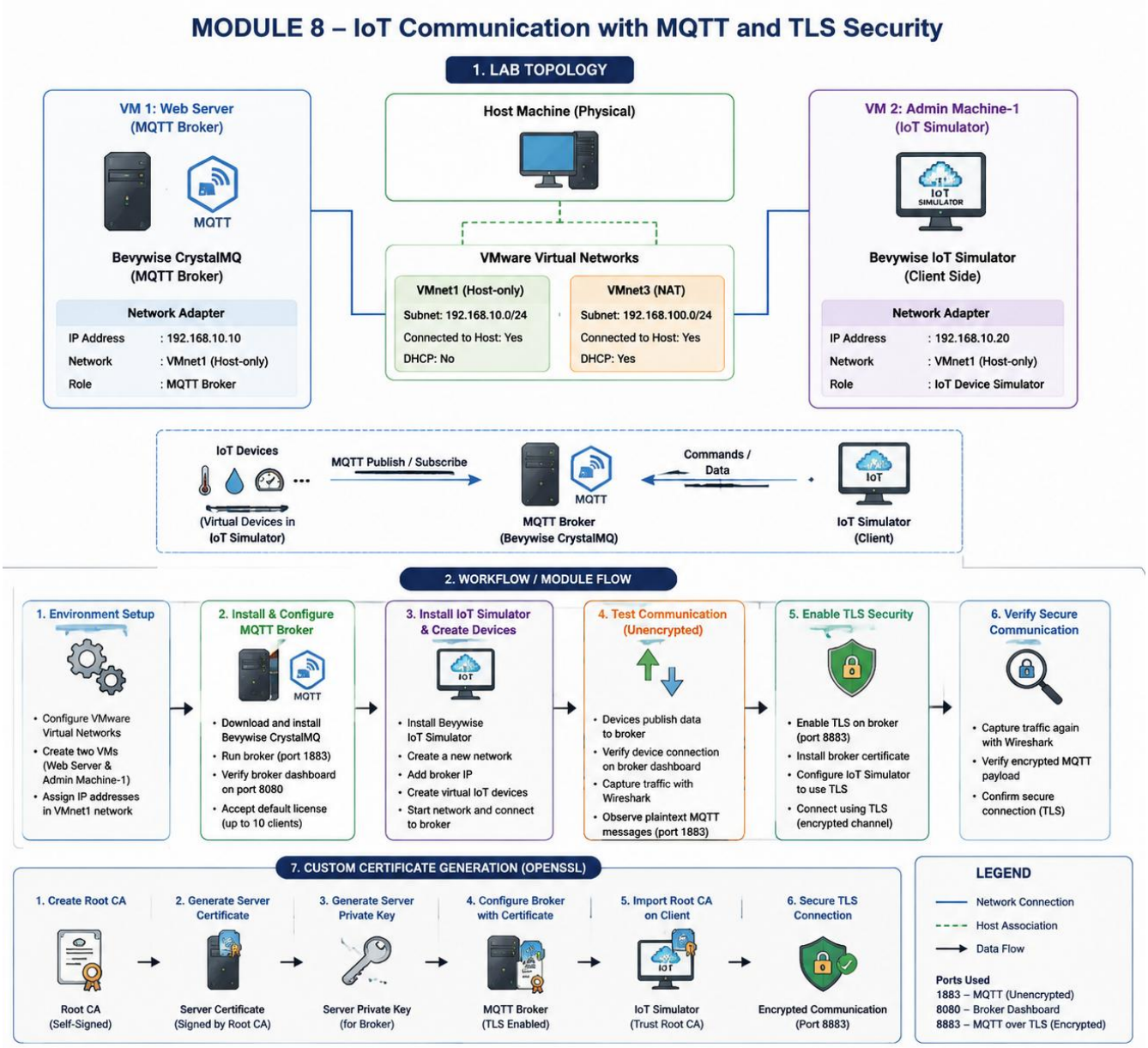
To improve communication security, **TLS encryption** was implemented on the MQTT Broker and IoT Simulator. A secure MQTT connection was configured using certificates, changing communication from standard MQTT over TCP port **1883** to encrypted MQTT communication over TLS port **8883**. Packet analysis after TLS implementation confirmed that MQTT payload data was encrypted and no longer readable, demonstrating the effectiveness of secure communication.

Furthermore, a custom Public Key Infrastructure (PKI) setup was created using OpenSSL by generating a self-signed Root Certificate Authority (CA), server certificate, and private key. These certificates were integrated into the MQTT environment to replace the default certificate configuration and verify secure authentication between the broker and client.

Overall, this module provided practical experience in IoT network architecture, MQTT communication, packet analysis, certificate management, and the implementation of encryption mechanisms to protect IoT data transmission.

Contents

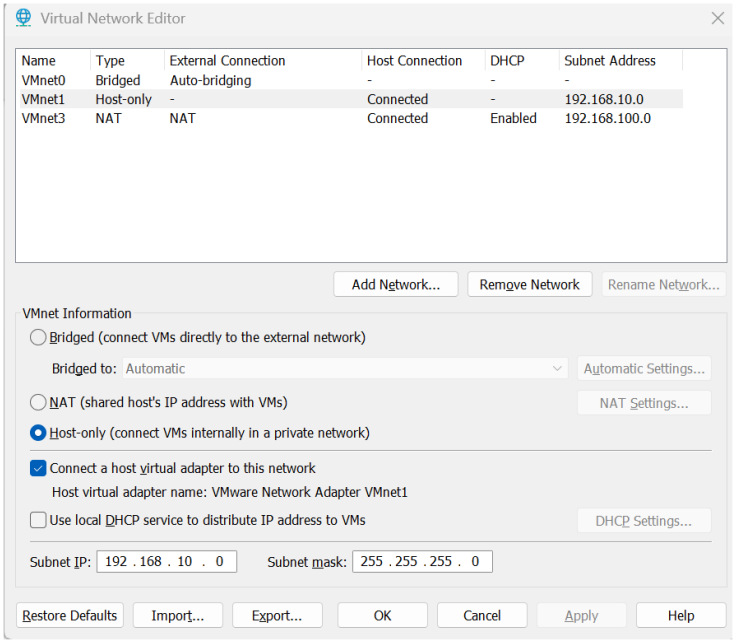
Virtual Network Editor Setting 5
Virtual Machine IP Configuration and Connectivity Testing 6
MQTT Broker Installation and Configuration (Web server) 8
IoT Simulator Installation and Network Creation 13
MQTT Communication Testing 15
Subscribe A Command Topic to IOT Simulator 19
Wireshark Plaintext Traffic Analysis 20
TLS Configuration and Secure MQTT Communication 24
Generate Your Own Self-Signed Certificate with OpenSSL 34
Conclusion 44

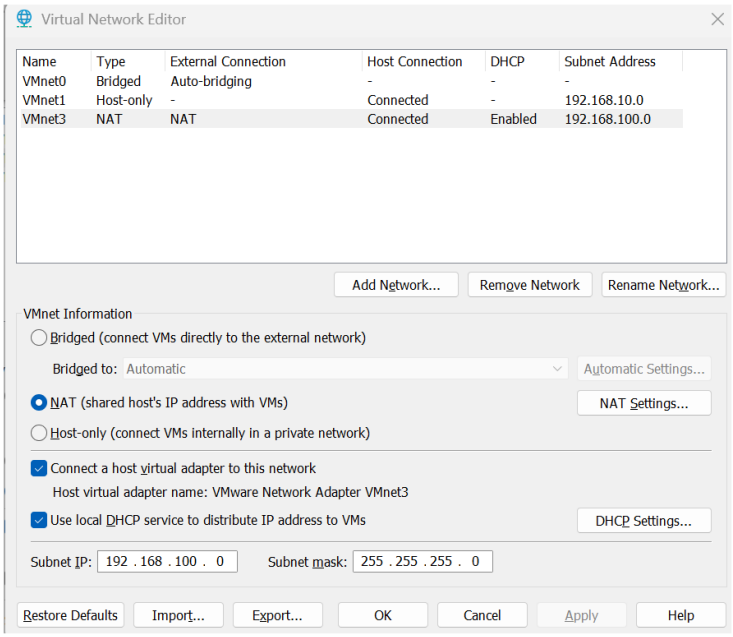

MODULE 8 – IoT Communication with MQTT and TLS Security
1. LAB TOPOLOGY
VM 1: Web Server (MQTT Broker)
MQTT
Bevywise CrystalMQ (MQTT Broker)
Network Adapter
IP Address : 192.168.10.10
Network : VMnet1 (Host-only)
Role : MQTT Broker
Host Machine (Physical)
VMware Virtual Networks
VMnet1 (Host-only)
Subnet: 192.168.10.0/24
Connected to Host: Yes
DHCP: No
VMnet3 (NAT)
Subnet: 192.168.100.0/24
Connected to Host: Yes
DHCP: Yes
VM 2: Admin Machine-1 (IoT Simulator)
IoT SIMULATOR
Bevywise IoT Simulator (Client Side)
Network Adapter
IP Address : 192.168.10.20
Network : VMnet1 (Host-only)
Role : IoT Device Simulator
IoT Devices
(Virtual Devices in IoT Simulator)
MQTT Publish / Subscribe
MQTT
MQTT Broker (Bevywise CrystalMQ)
Commands / Data
IoT
IoT Simulator (Client)
2. WORKFLOW / MODULE FLOW
1. Environment Setup
Configure VMware Virtual Networks
Create two VMs (Web Server & Admin Machine-1)
Assign IP addresses in VMnet1 network
2. Install & Configure MQTT Broker
MQTT
Download and install Bevywise CrystalMQ
Run broker (port 1883)
Verify broker dashboard on port 8080
Accept default license (up to 10 clients)
3. Install IoT Simulator & Create Devices
IoT
Install Bevywise IoT Simulator
Create a new network
Add broker IP
Create virtual IoT devices
Start network and connect to broker
4. Test Communication (Unencrypted)
Devices publish data to broker
Verify device connection on broker dashboard
Capture traffic with Wireshark
Observe plaintext MQTT messages (port 1883)
5. Enable TLS Security
Enable TLS on broker (port 8883)
Install broker certificate
Configure IoT Simulator to use TLS
Connect using TLS (encrypted channel)
6. Verify Secure Communication
Capture traffic again with Wireshark
Verify encrypted MQTT payload
Confirm secure connection (TLS)
7. CUSTOM CERTIFICATE GENERATION (OPENSSL)
1. Create Root CA
Root CA (Self-Signed)
2. Generate Server Certificate
Server Certificate (Signed by Root CA)
3. Generate Server Private Key
Server Private Key (for Broker)
4. Configure Broker with Certificate
MQTT Broker (TLS Enabled)
5. Import Root CA on Client
IoT
IoT Simulator (Trust Root CA)
6. Secure TLS Connection
Encrypted Communication (Port 8883)
LEGEND
Network Connection
Host Association
Data Flow
Ports Used
1883 – MQTT (Unencrypted)
8080 – Broker Dashboard
8883 – MQTT over TLS (Encrypted)

Virtual Network Editor Setting

The VMware Virtual Network Editor is used to configure the virtual networks (VMnets) used by the lab VMs.

- **VMnet1** — Host-only network, Subnet: 192.168.10.0, connected to host, no DHCP.
- **VMnet3** — NAT network, Subnet: 192.168.100.0, DHCP enabled, connected to host.

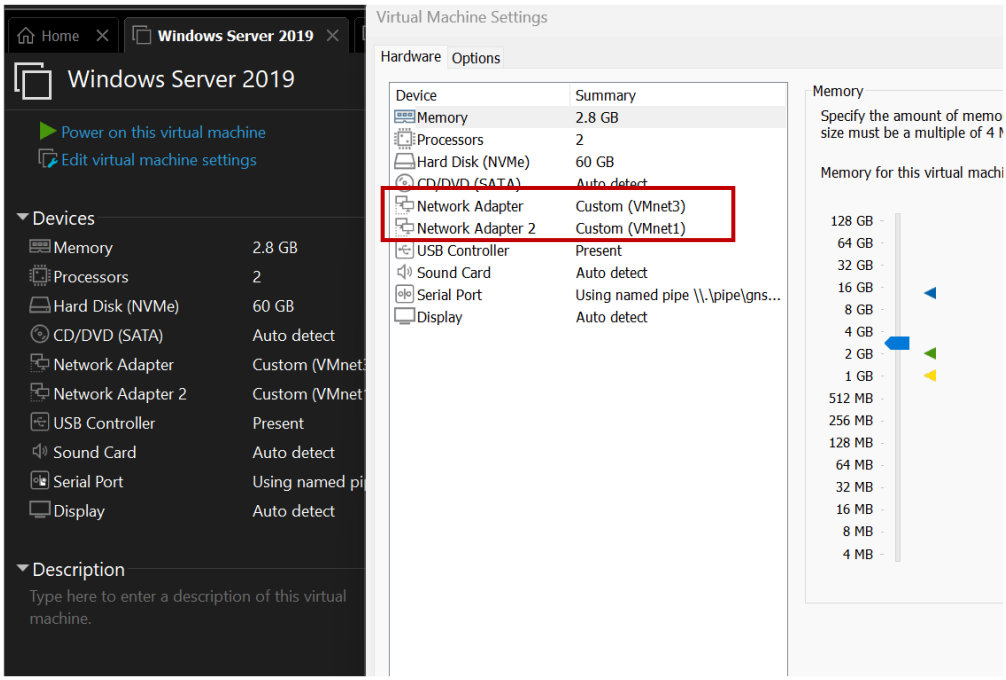




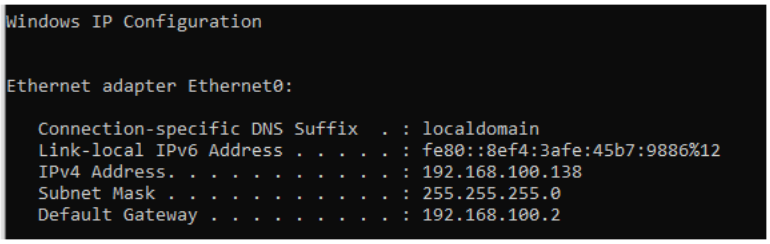
Module 8 requires two virtual machines (VMs):

1. **Web Server –** The MQTT Broker (Bevywise MQTTRoute) will be installed on this machine. It acts as the server-side component responsible for collecting and managing data from IoT devices.
2. **Admin Machine-1 –** The IoT Simulator will be installed on this machine. It acts as the client-side component, where virtual IoT devices will be created and connected to the MQTT Broker.

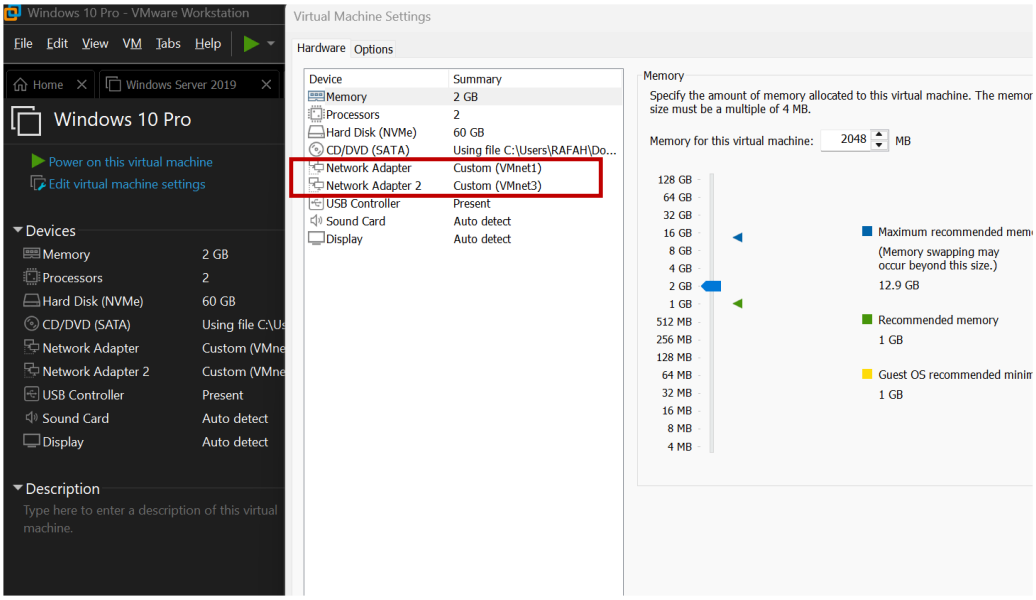
Virtual Machine IP Configuration and Connectivity Testing



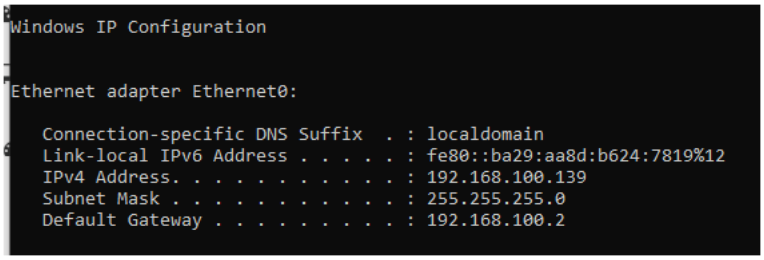
IP check



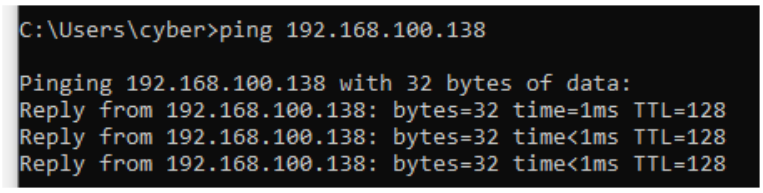
Admin Machine



IP check

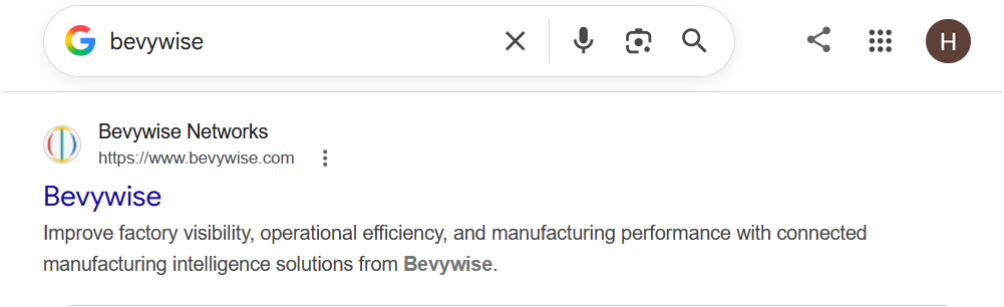


Connectivity check

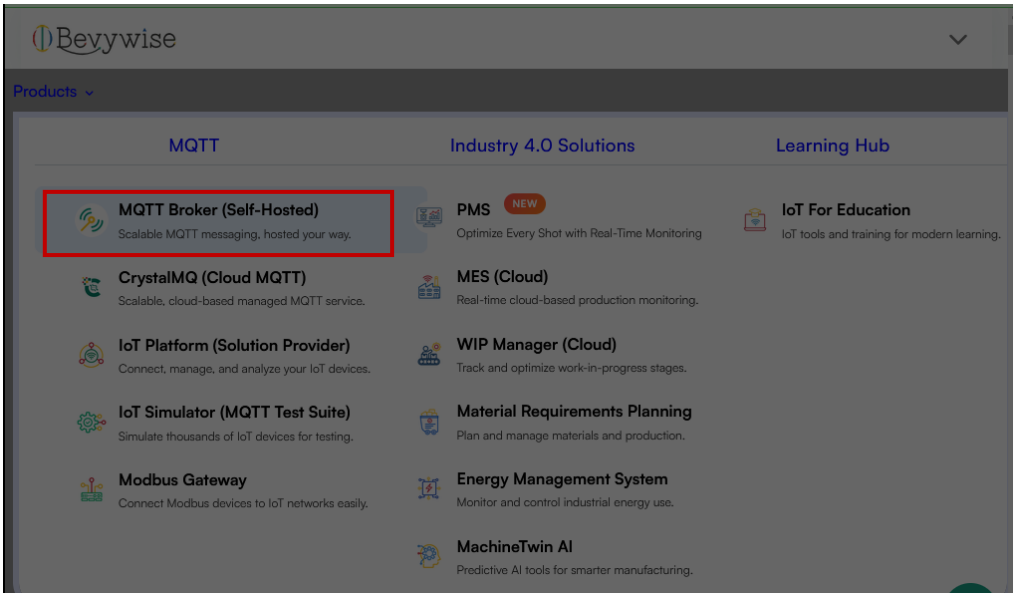


MQTT Broker Installation and Configuration (Web server)

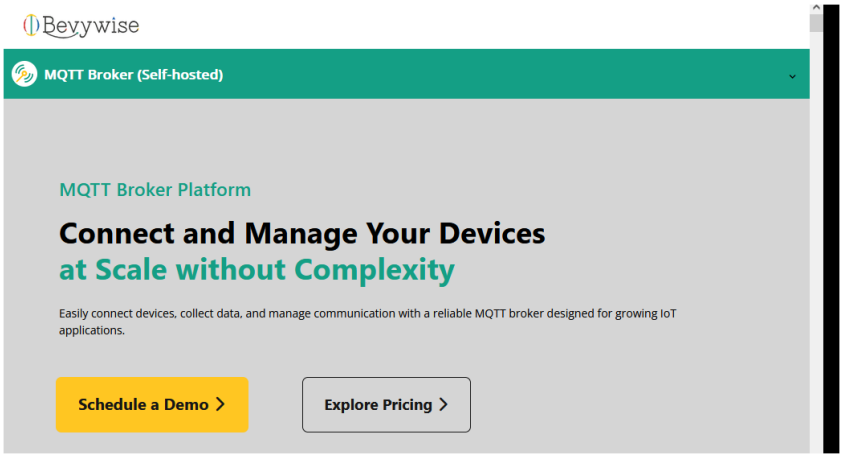
- Open one of your windows (that surely have admin rights)
- Navigate to the bavywise website



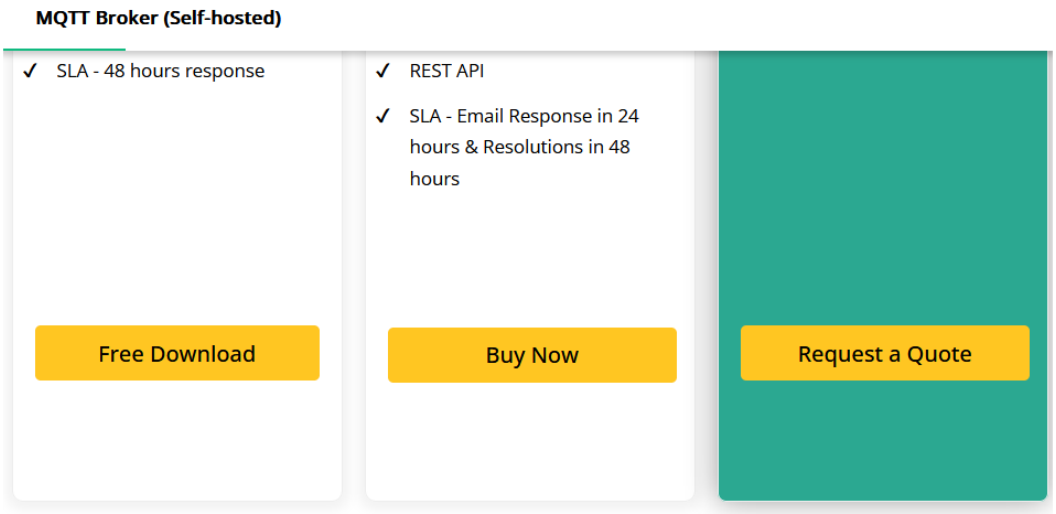
Click on the right-side icon a dropdown menu will open select MQTT broker



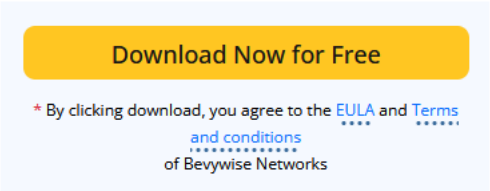
Click on explore pricing



Scroll down and select free download

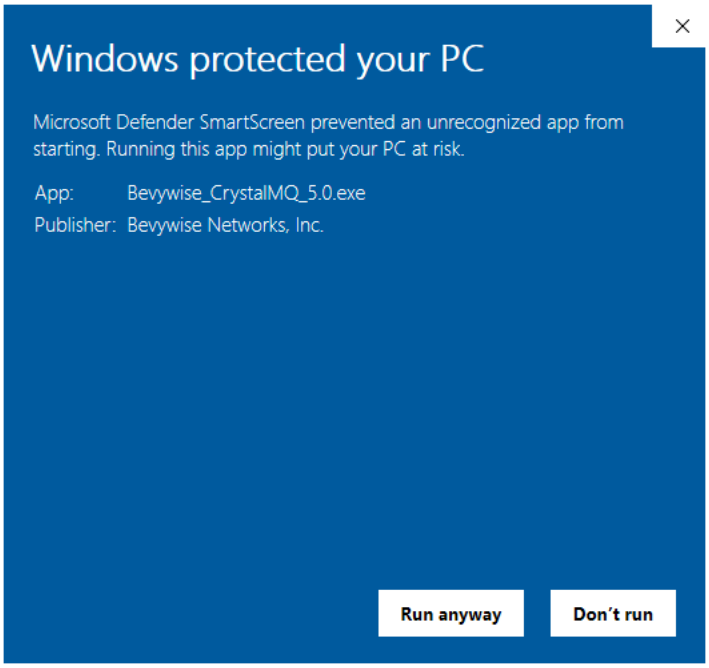


Enter credentials and click on download now for free

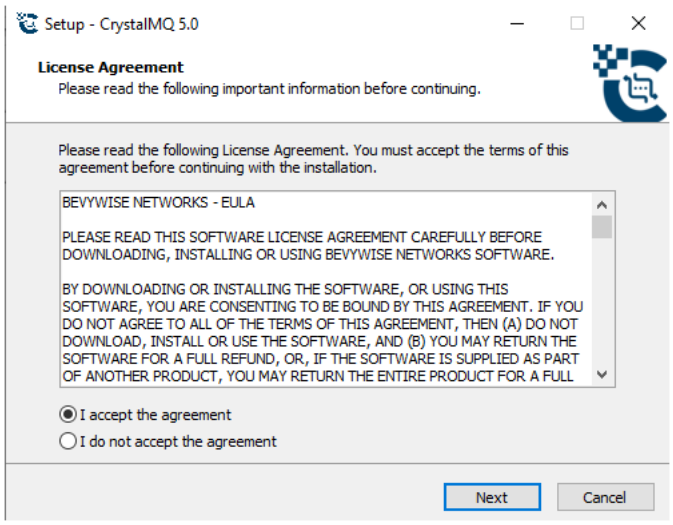


Wait for the installation to complete….

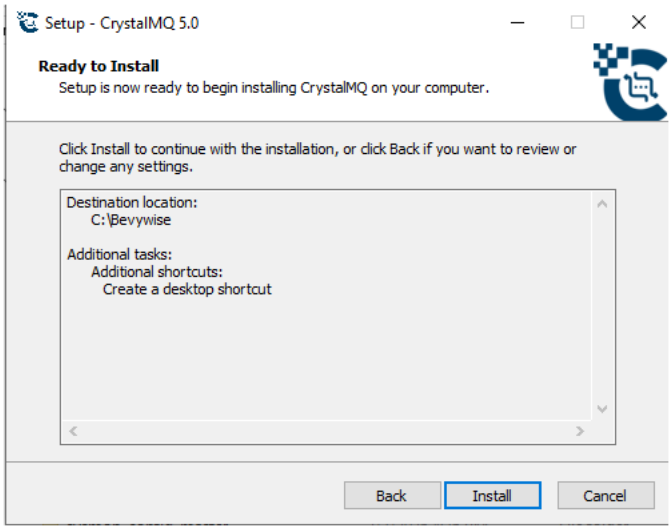
Launch the Setup (Turn off Windows Security)



Accept the terms

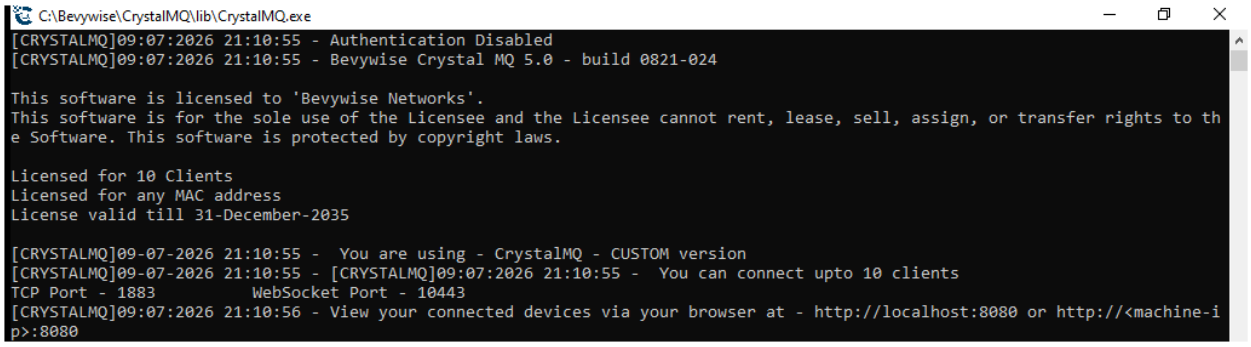


Proceed installation

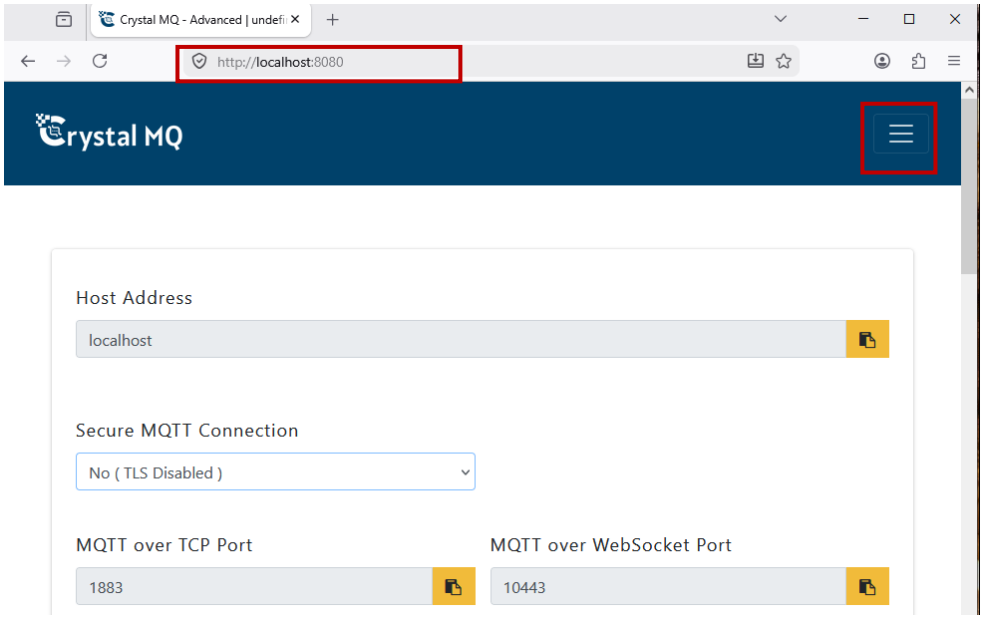


MQTT Broker installed & Running Crystal MQ

- Authentication Disabled — no login required for devices to connect
- Bevywise Crystal MQ 5.0 (build 0821-024) — the broker software and version
- Licensed for 10 Clients — free version allows up to 10 devices
- Licensed for any MAC address — no hardware restriction
- License valid till 31-December-2035 — license expiry
- CrystalMQ - CUSTOM version — edition type
- Connect upto 10 clients — client limit confirmation
- TCP Port - 1883 — port for MQTT device connections
- WebSocket Port - 10443 — alternate WebSocket connection port
- Dashboard URL: http://localhost:8080 or http://<machine-ip>:8080 — web interface to view connected devices

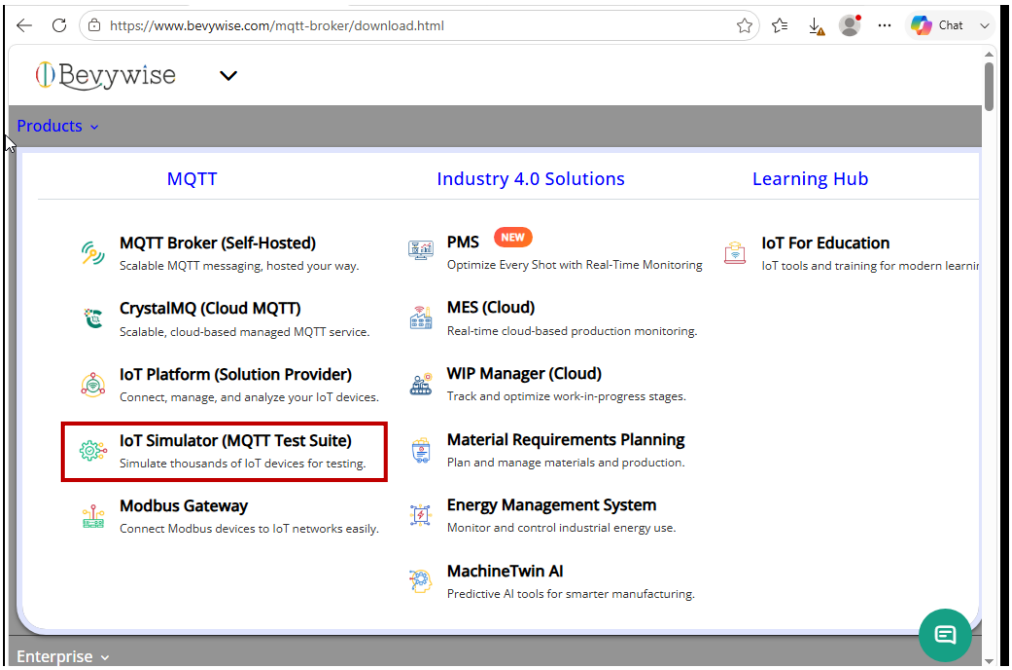


Now browse the given URL and see Crystal MQ running

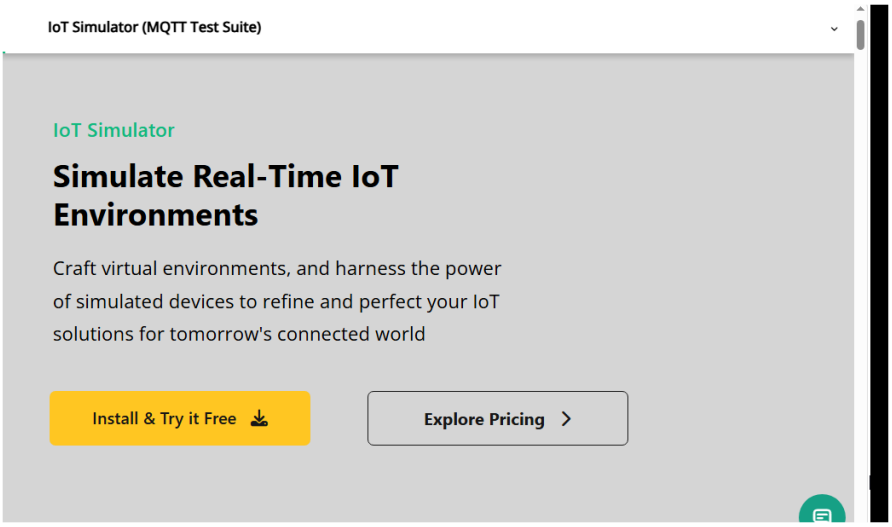


IoT Simulator Installation and Network Creation

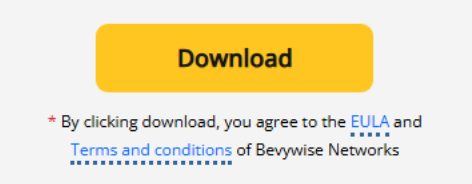
- Navigate to Product Select IOT Simulator (MQTT Test Suite)



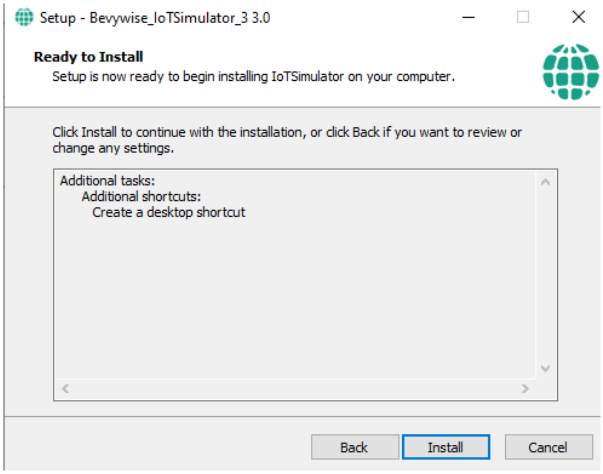
Select Install & Try it for Free



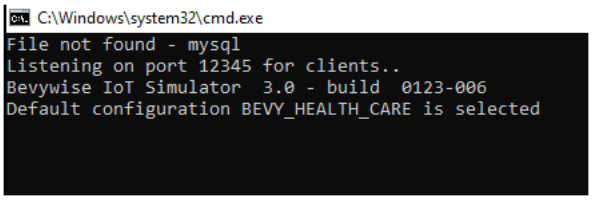
Enter the credentials and click on download



Launch Bevywise_Iot_Simulator

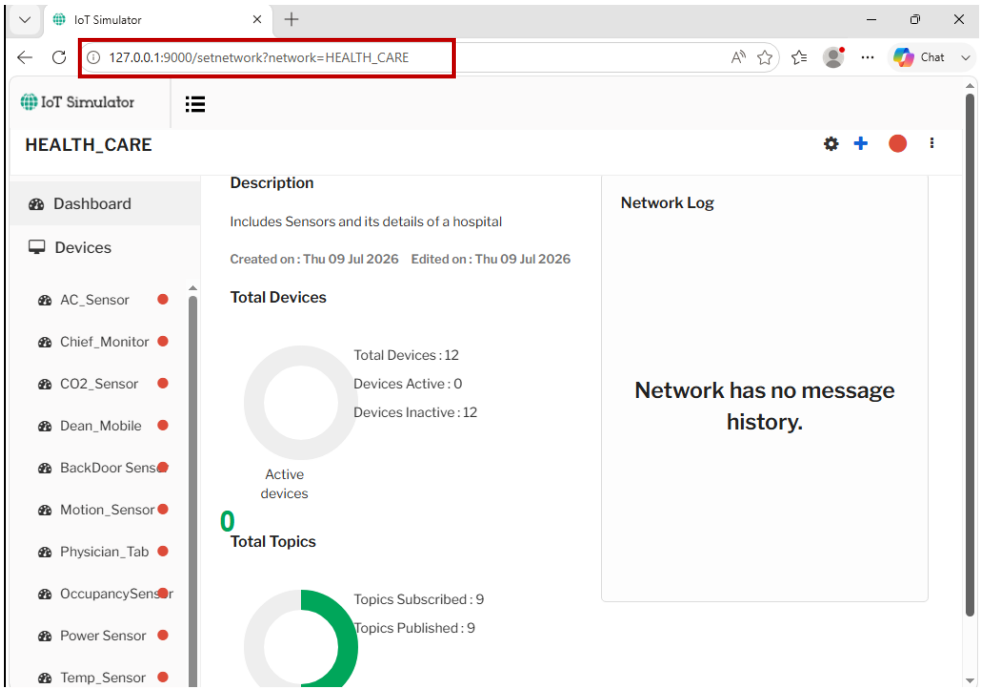


IOT Simulator loaded successfully

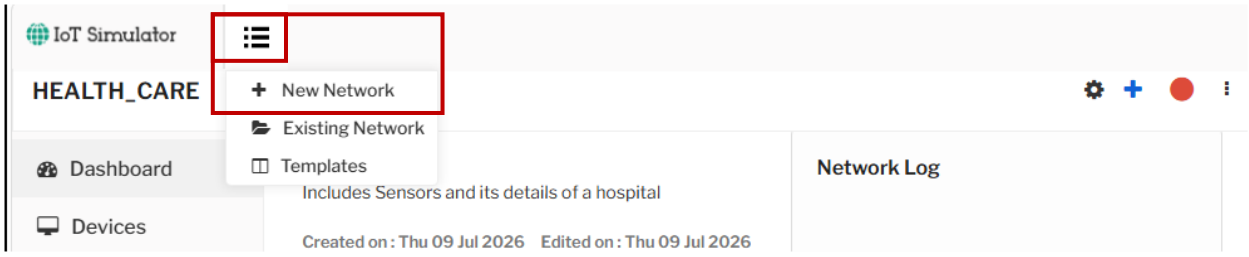


MQTT Communication Testing

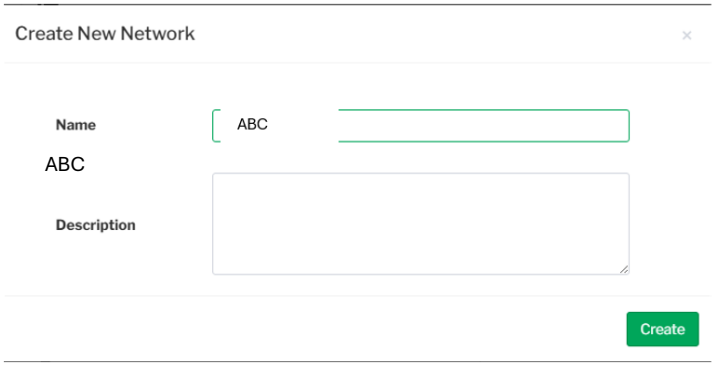
Dashboard showing default demo network **"HEALTH_CARE"** with (12 pre-made devices)



Setup a new network

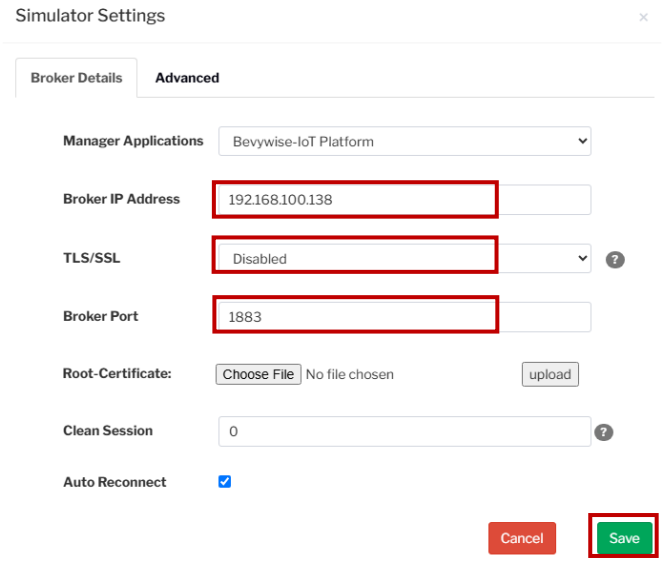


Add a name and click on create

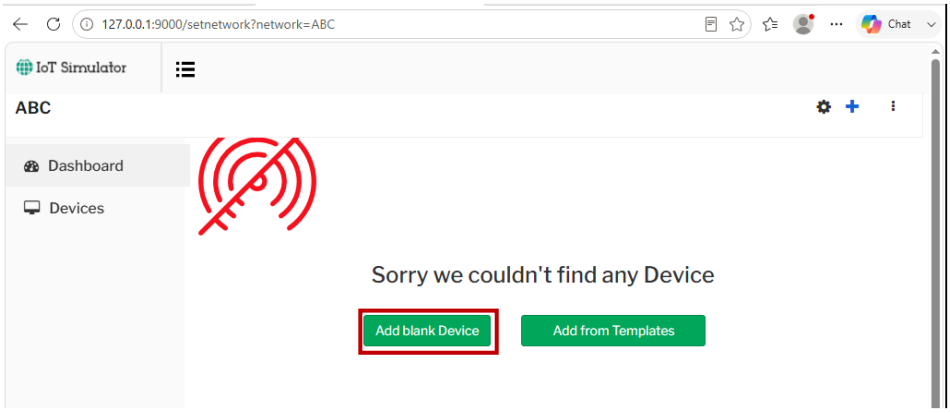


Now enter the IP of your VM machine which Is being used as a broker (IP address of the Web Server that is running Crystal MQ)

- Click save

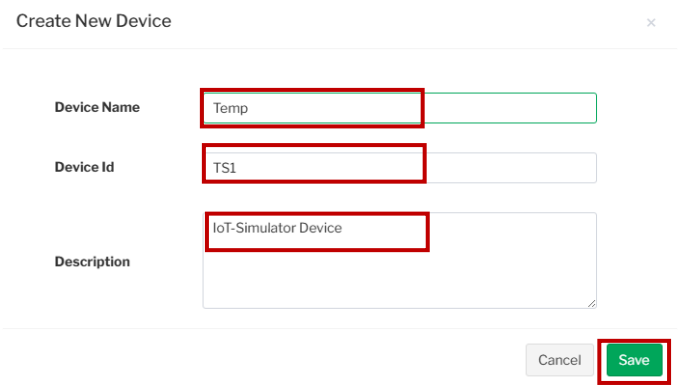


Created now click on add blank device



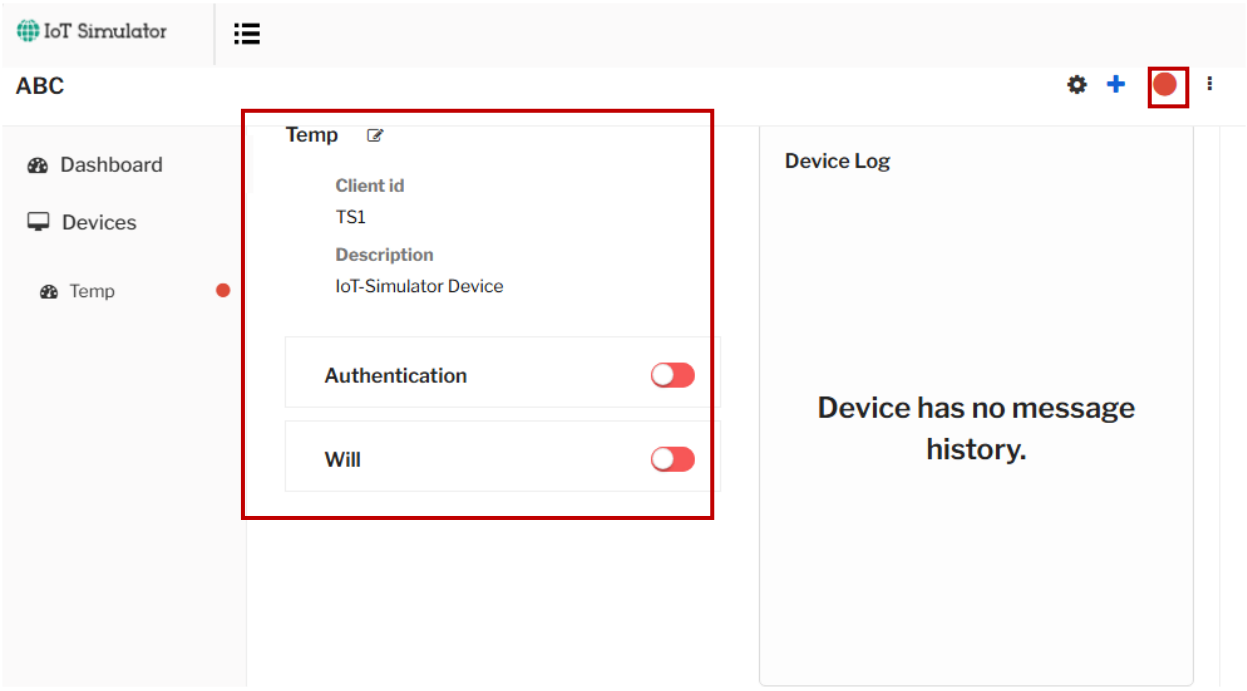
Add details for device:

- Name: Temp
- Id: TSI
- Description: IOT-Simulator Device

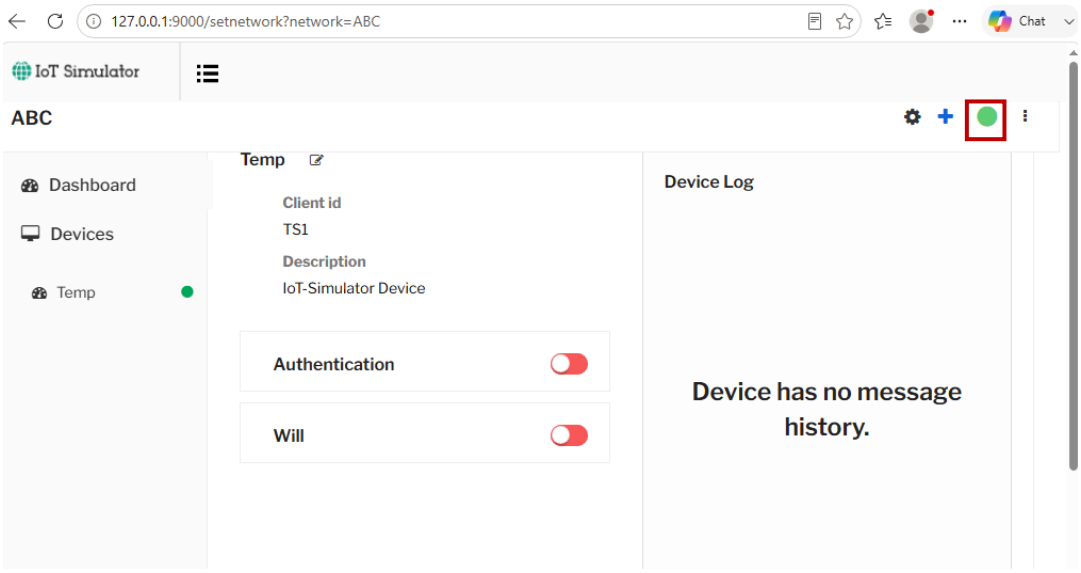


Device added successfully

Select the red circle and start the network (Make sure your broker is running on the other VM)

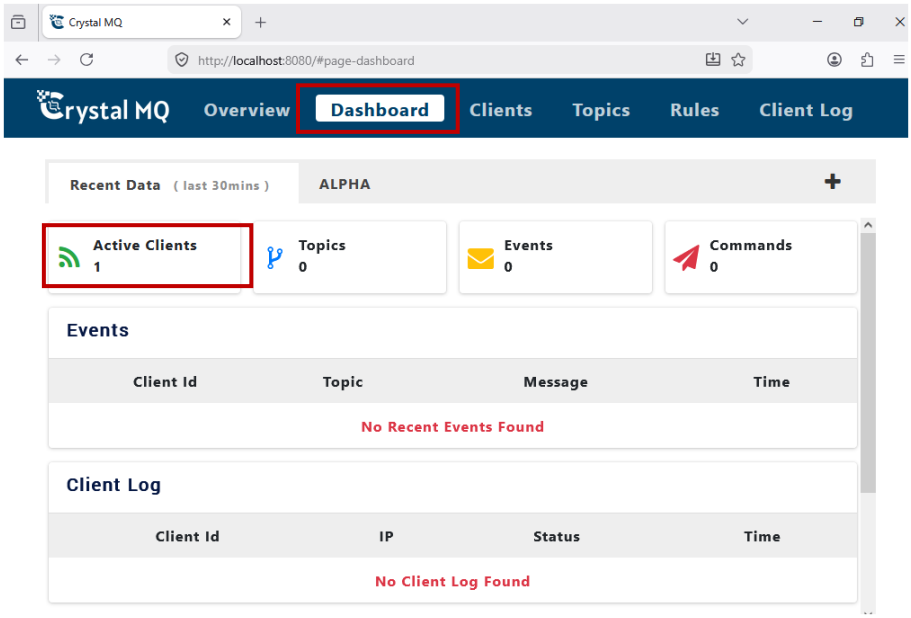


The red circle got green showing connected to broker



Navigate to Crystal MQ (Web Server) & check the dashboard on http://localhost:8080

- (Client is added)



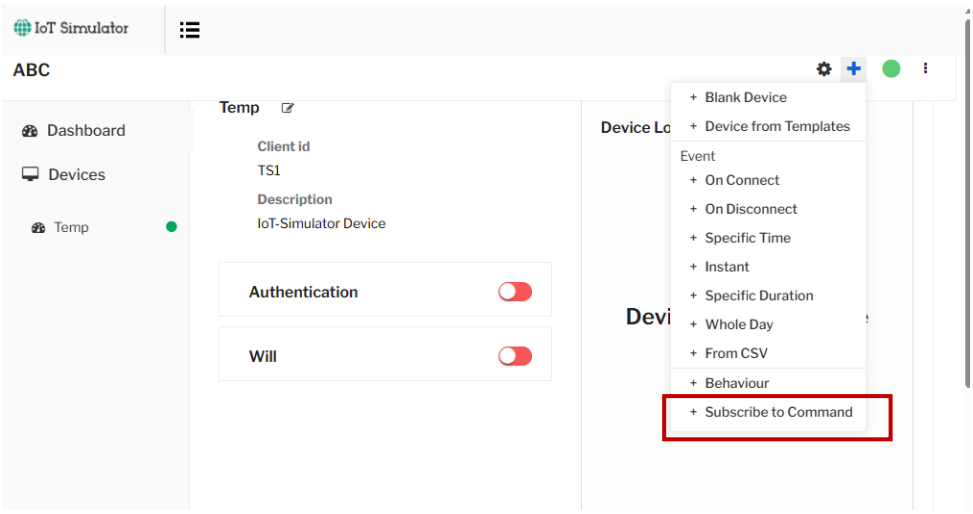
Checking the connections



Subscribe A Command Topic to IOT Simulator

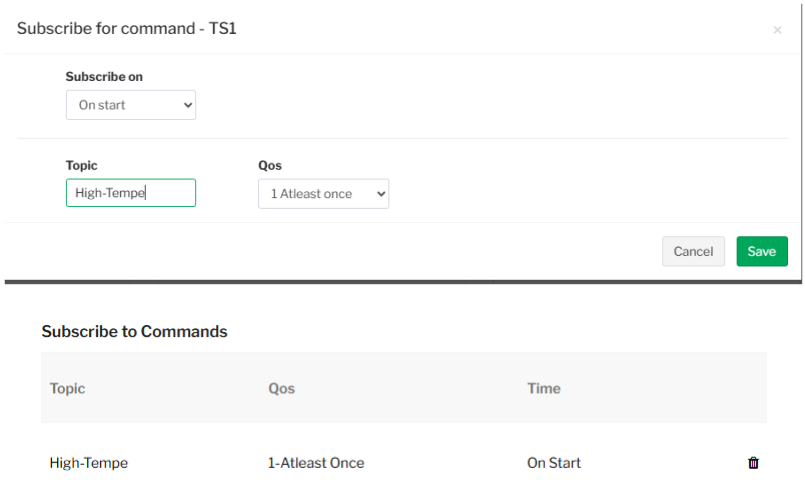
Now moving back to the IOT simulator (Admin VM)

Select the + icon to Subscribe to a Command Topic



Command set for high temperature

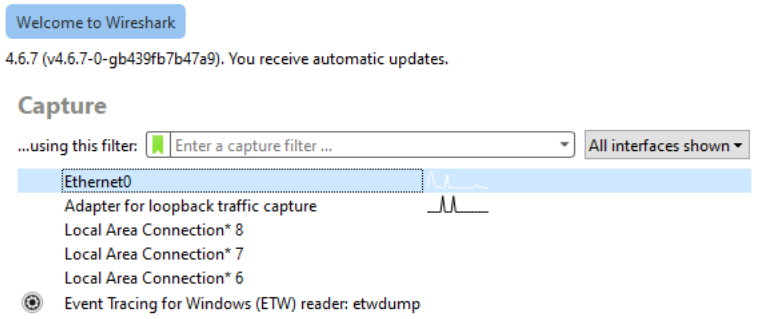
Command = High-Tempe

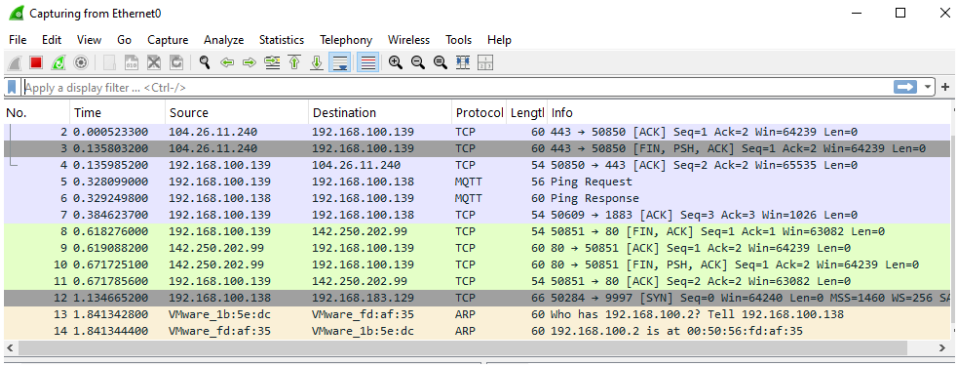


Wireshark Plaintext Traffic Analysis

Open Wireshark on Admin-VM

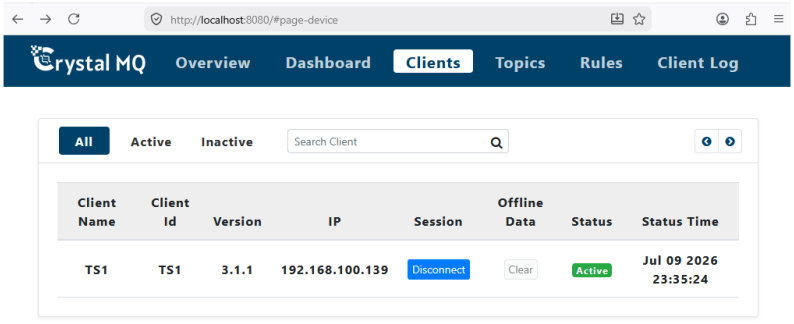
- Start capture
- Stay this on Capture state





Open Web Server -> CRYSTAL MQ dashboard(http://localhost:8080)

- send a command towards the Admin VM (iot simulator)
- Select client and click on the client TS1

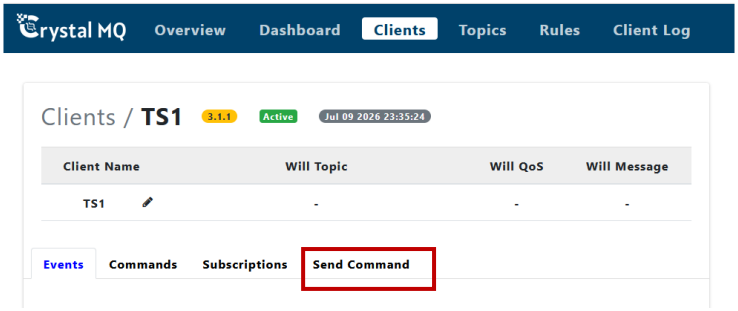


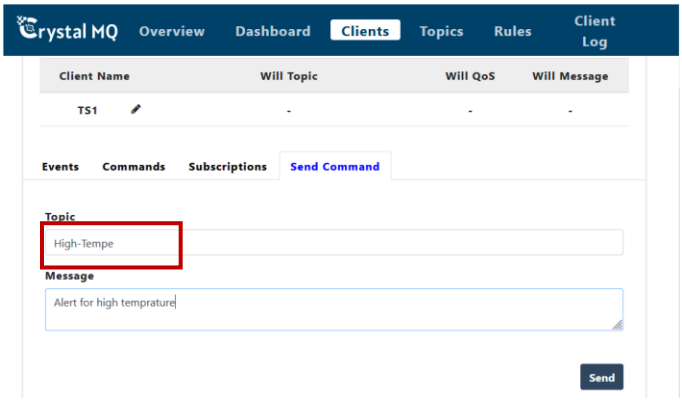
Now select send commands

Enter here the command you made before

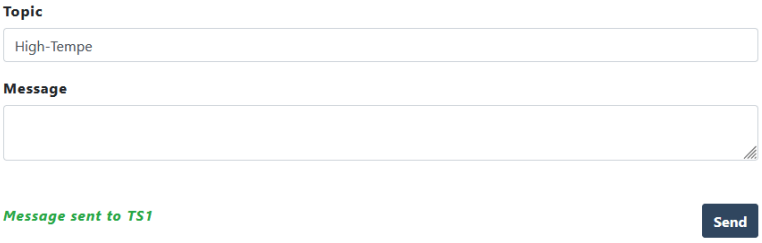
Command = High-Tempe

Click on send to send the command



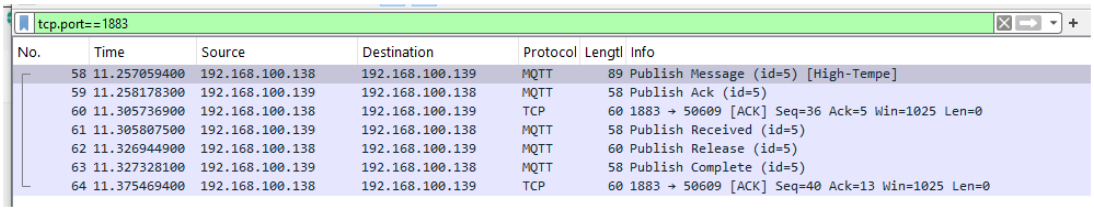


Command Worked Successfully

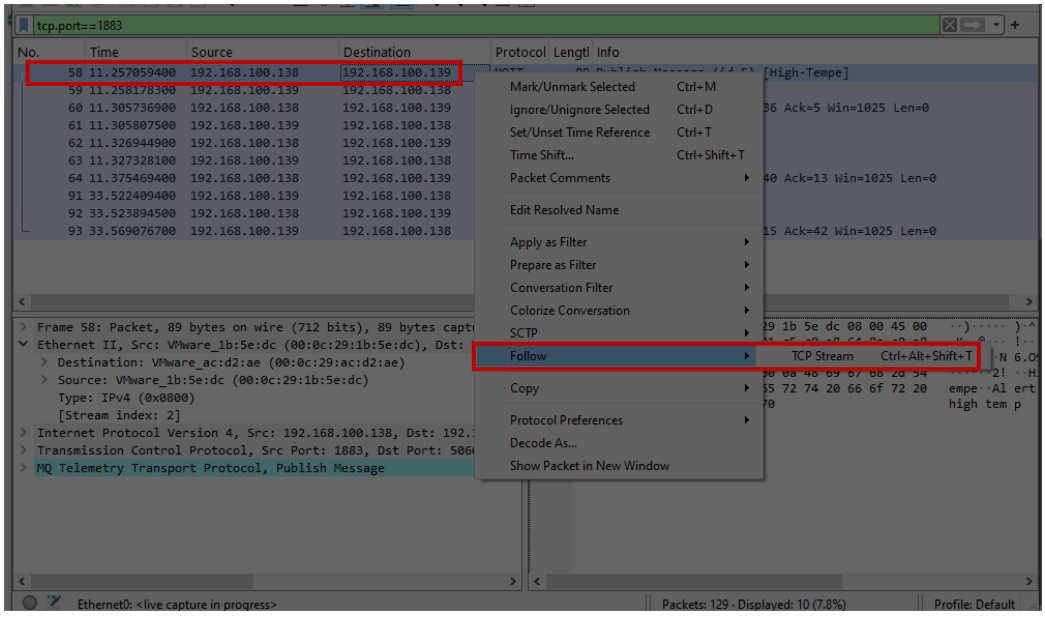


Open Wireshark in Admin-VM (already running)

Filter: tcp.port==1883

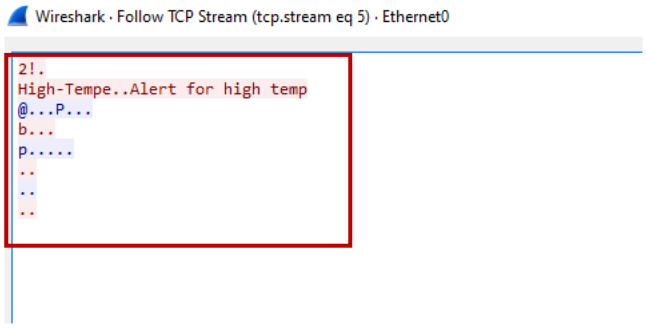


Right click on the selected one and select follow/tcp stream



This is your **plaintext MQTT evidence** — the topic name and message are fully readable, proving that without encryption, anyone capturing this traffic (a man-in-the-middle attacker) could read sensitive IoT data in clear text.

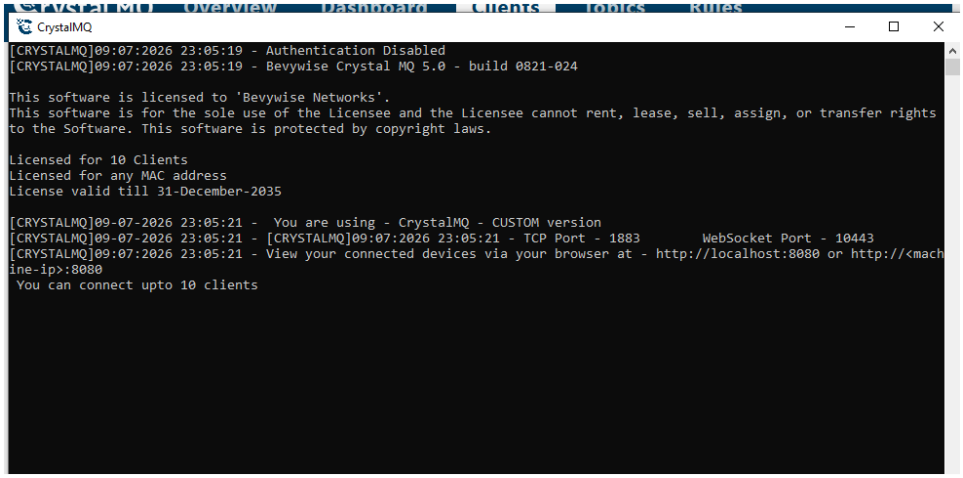
Save/keep this screenshot — it's your "before TLS" proof for the lab report.



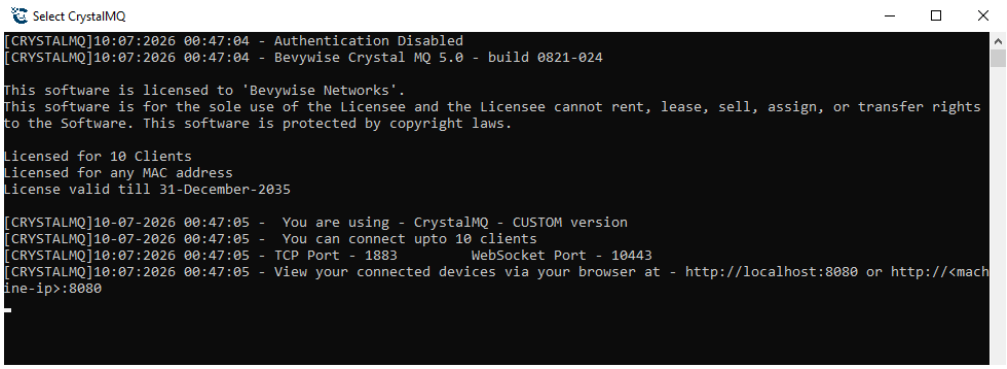
TLS Configuration and Secure MQTT Communication

Now come back to web server (broker side)

- Stop CRYSTAL MQ

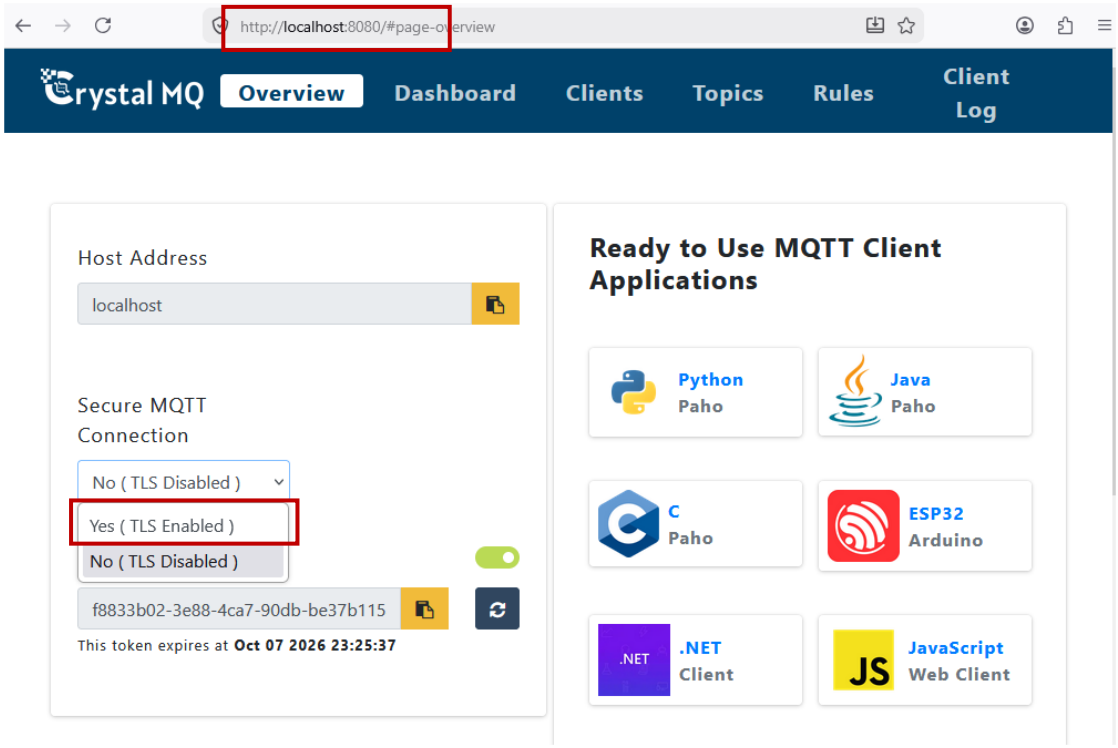


Run Crystal MQ

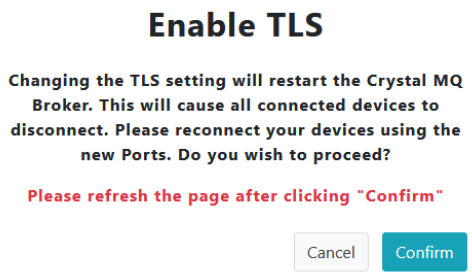


Now use the URL to open the web interface http://localhost:8080

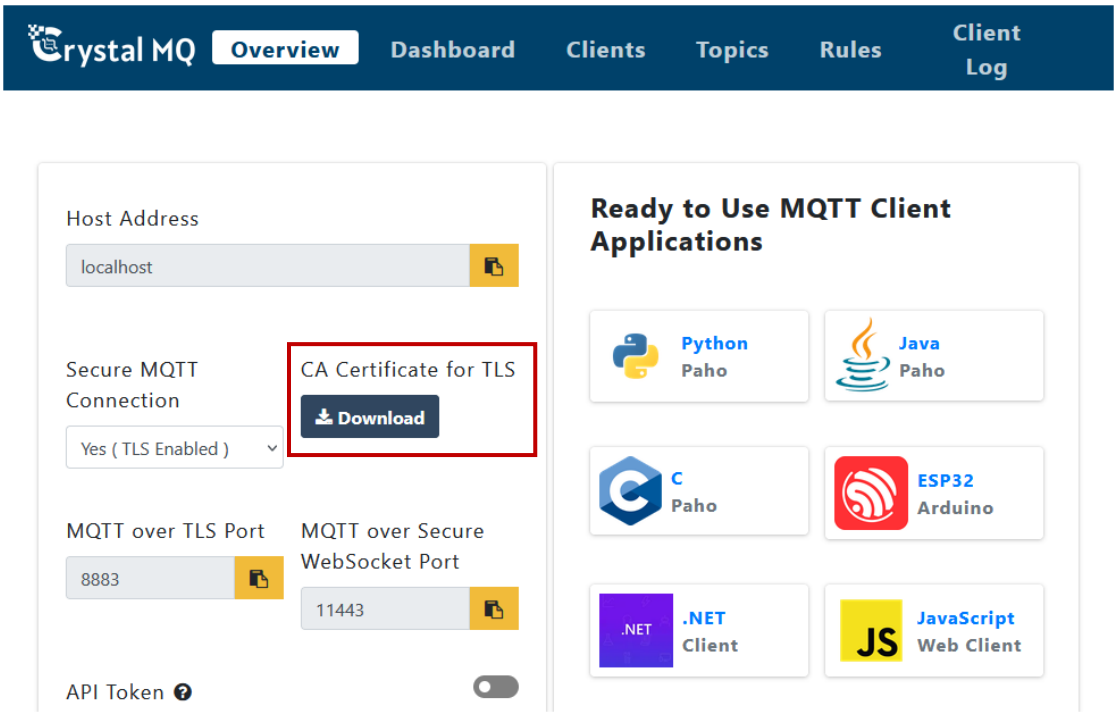
- Now select and enable TLS



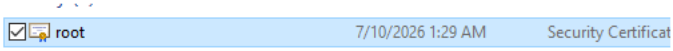
Click confirm



Now close the Web server and open it again & click on download to get the certificate which will be used for TLS Handshake

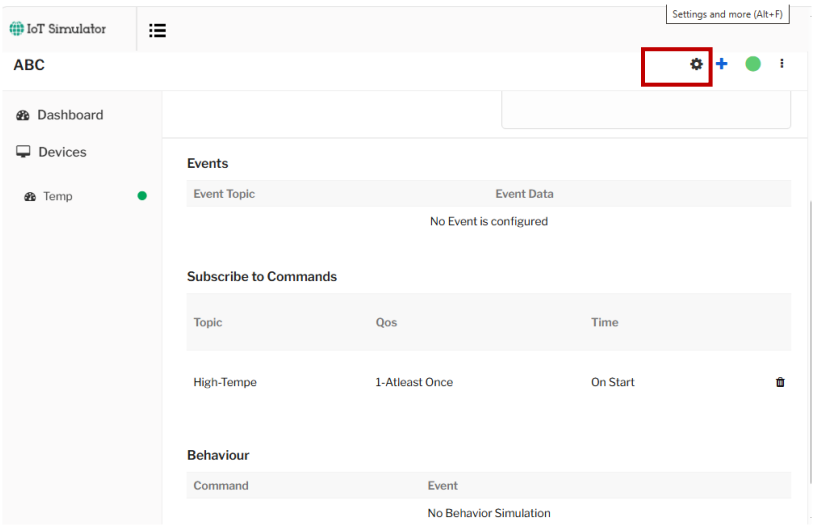


Certificate is downloaded



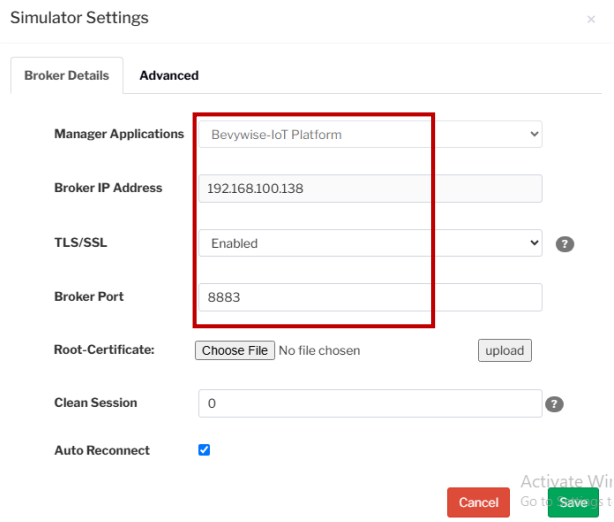
Open Admin VM (iot-simulator)

- Click on Settings icon

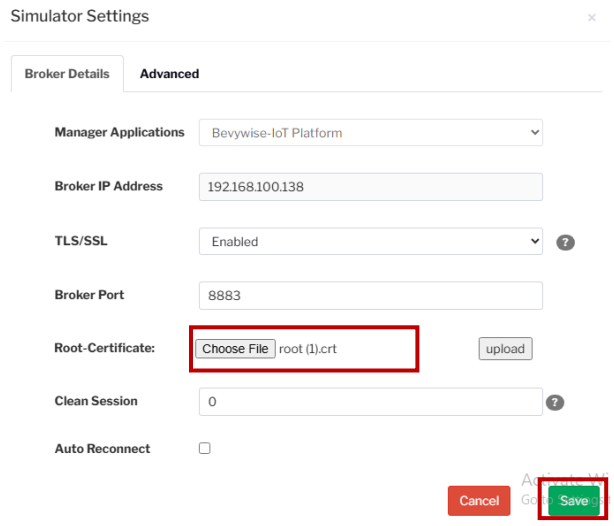


Select TLS enable & make sure the port is same as we see in the Crystal MQ (Broker side)

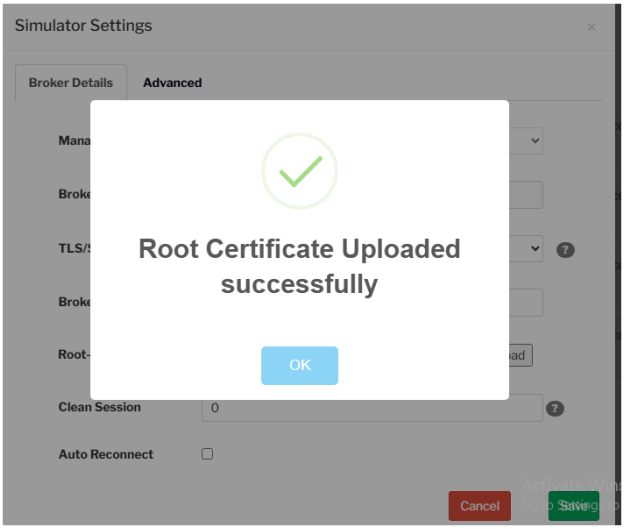
- Attach the certificate we downloaded in the Web Server (Crystal MQ)



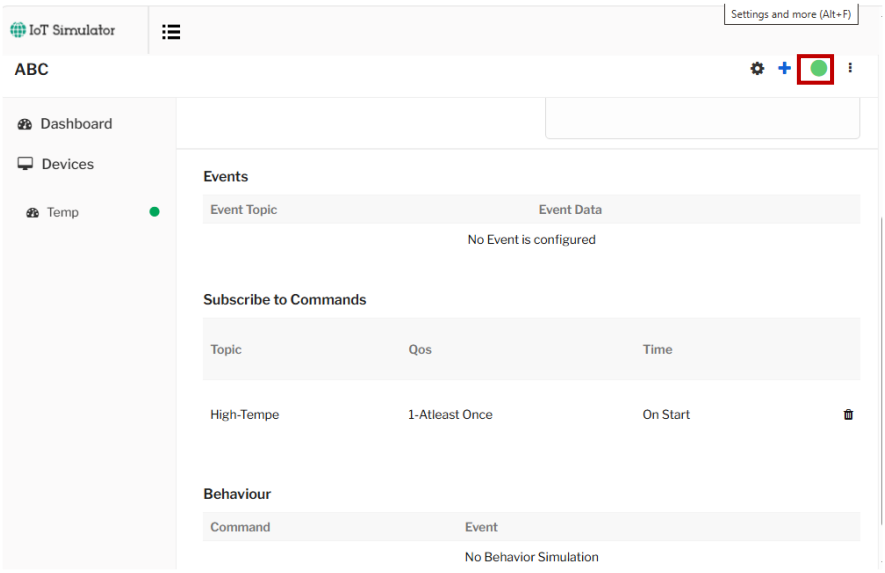
Certificate Added



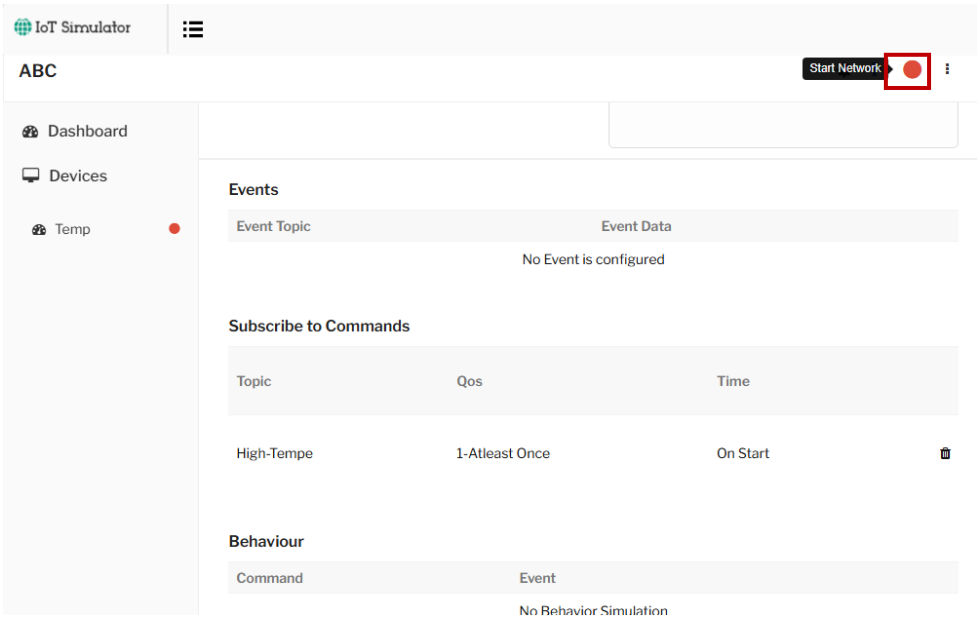
Updated Successfully



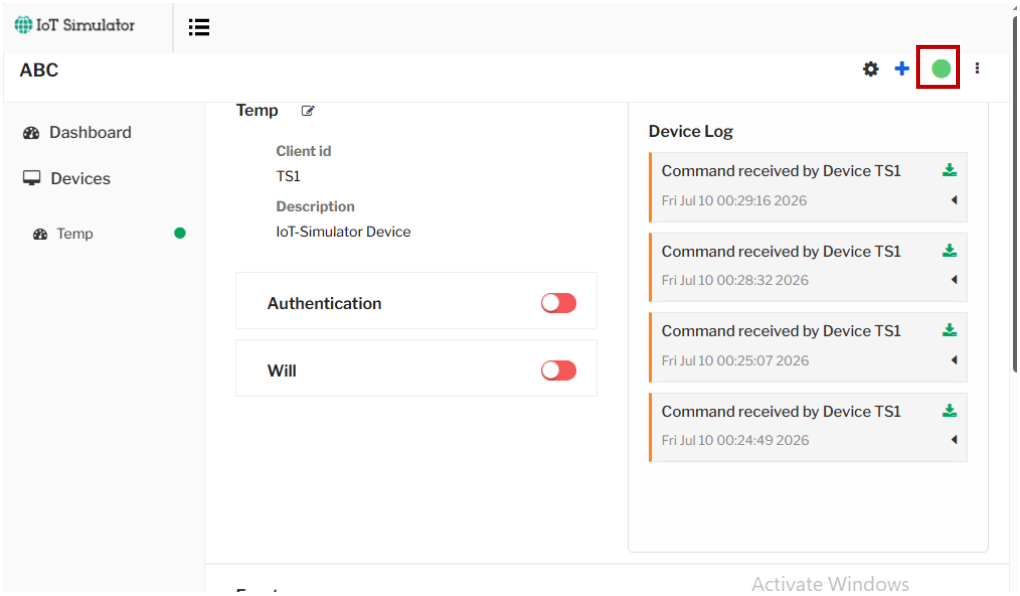
Now click on the green circle it will stop the network



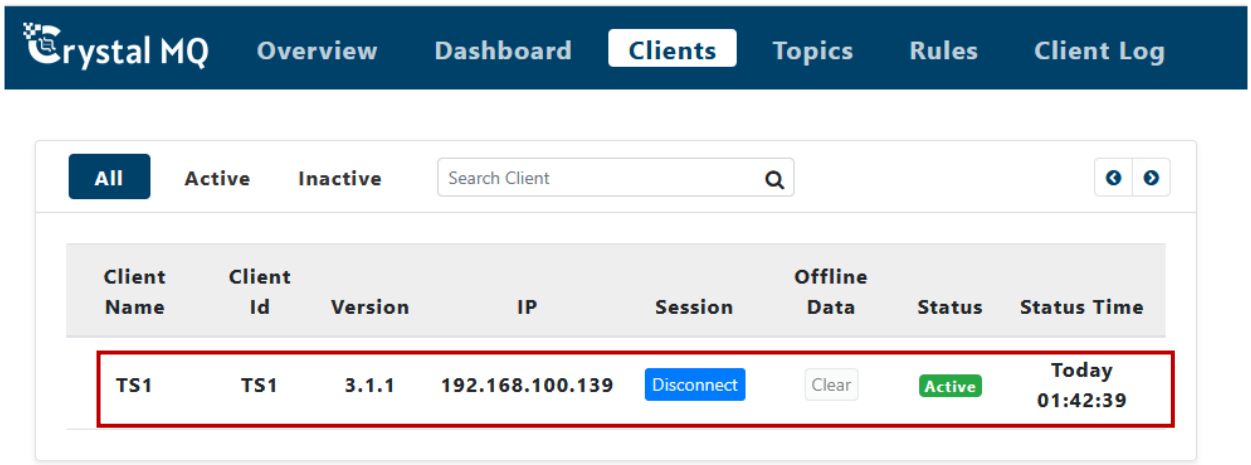
Now again click on the red circle to connect It to the web server



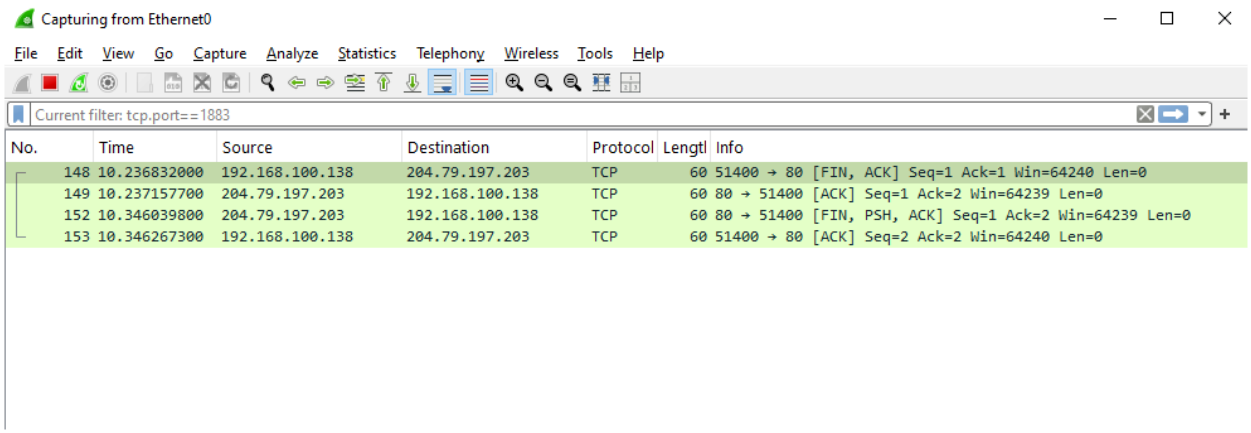
Network has been started
let's check whether its connection is ok with broker side or not



Broker side configuration cleared that the connection is established clearly

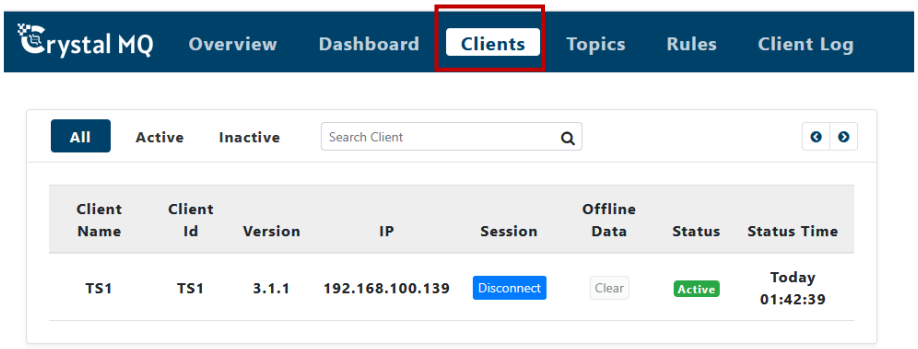


Open Wireshark in Admin VM (Running in Capture State)

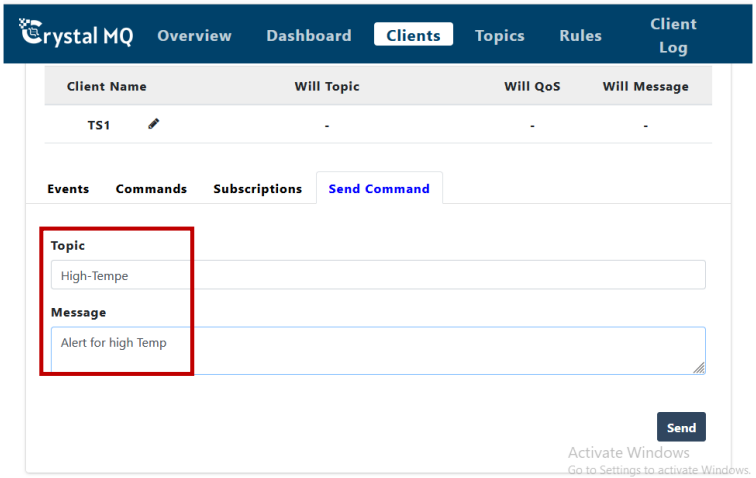


Now move to the broker side (Web server) and send commands towards the Admin VM (iot simulator)

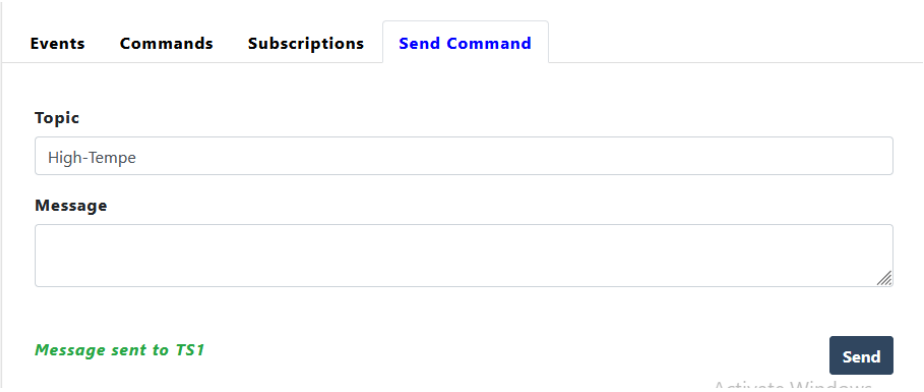
Select Client



Send command from broker

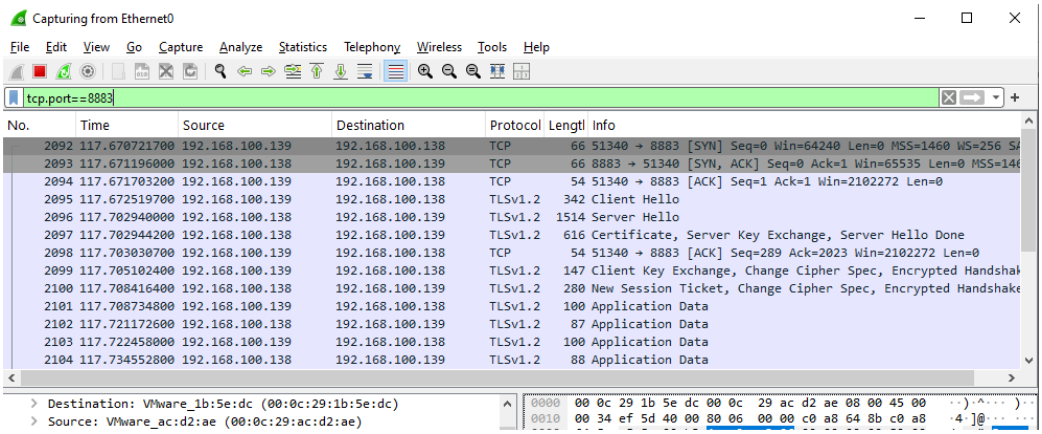


Command sends successfully



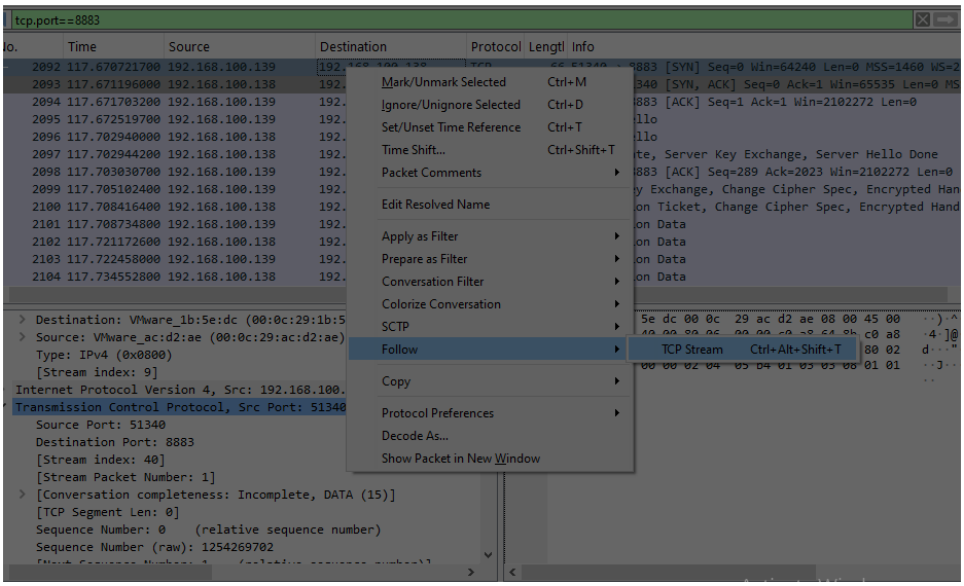
Wireshark Checking Packets

Apply this filter tcp.port==8883



Select the first one as it shows the Ip of admin VM

Right click on it
Select follow /TCP Stream



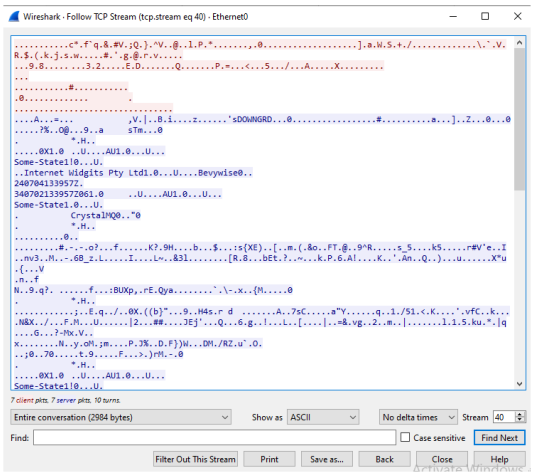
TLS is working perfectly!

Look at that TCP Stream output — it's completely unreadable garbage/binary data (.c*.fq.&.#V.;Q.}.^V.@..1.P.*` etc.), with just certificate metadata visible in plaintext (Common Name, Organization "Internet Widgits Pty Ltd", validity dates — that's normal, it's the TLS handshake certificate exchange, not your actual message).

Compare this to your earlier plaintext capture where you could clearly read:

High-Tempe..Alert for high temp

Now your actual MQTT payload (topic + message) is **fully encrypted** — completely unreadable to anyone sniffing the traffic. This is your **"after TLS" proof**, exactly matching what the lab manual demonstrates (Step 100-101 showing TLSv1.2 Application Data with encrypted content).



Generate Your Own Self-Signed Certificate with OpenSSL

Right now you're using CrystalMQ's auto-generated certificate. Let's replace it with one you create yourself — this teaches you the actual PKI chain (Root CA → Server Cert → Private Key) instead of just clicking a button.

Install openssl on your web server

winget install ShiningLight.OpenSSL.Light

```
C:\Users\cyber>winget install ShiningLight.OpenSSL.Light
The `msstore` source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function prope
rly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes  [N] No: yes
Found OpenSSL Light [ShiningLight.OpenSSL.Light] Version 4.0.1
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
This package requires the following dependencies:
  - Packages
      Microsoft.VCRedist.2015+.x64
(1/1) Found Microsoft Visual C++ v14 Redistributable (x64) [Microsoft.VCRedist.2015+.x64] Version 14.51.36247.0
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://download.visualstudio.microsoft.com/download/pr/ebdab8e5-1d7b-4d9f-a11b-cbb1720c3b12/843068991DAAA1F
A73AD9F6239BCE4D0F6A07A51F18C37EA2A867E9BECA71295C/VC_redist.x64.exe
  17.8 MB / 17.8 MB
Successfully verified installer hash
```

Finding the location where openssl has been installed

```
C:\Users\cyber>where /R "C:\Program files" openssl.exe
C:\Program files\OpenSSL-Win64\bin\openssl.exe
C:\Program files\SplunkUniversalForwarder\bin\openssl.exe
C:\Program files\SplunkUniversalForwarder\opt\openssl1\bin\openssl.exe
```

Version verified

```
C:\Users\cyber>"C:\Program files\SplunkUniversalForwarder\opt\openssl1\bin\openssl.exe" version
WARNING: can't open config file: ::::::/openssl.cnf
OpenSSL 1.0.2zm-fips  30 Sep 2025
```

Step 2: Create a working folder for your custom certs

```
C:\Users\cyber>mkdir C:\MyCerts

C:\Users\cyber>cd C:\MyCerts

C:\MyCerts>
```

Now set your path so you can access openssl

```
C:\MyCerts>set PATH=%PATH%;C:\Program files\OpenSSL-Win64\bin

C:\MyCerts>openssl version
OpenSSL 4.0.1 9 Jun 2026 (Library: OpenSSL 4.0.1 9 Jun 2026)
```

Step 3: Generate a Root CA (self-signed)

```
C:\MyCerts>openssl genrsa -out rootCA.key 2048
```

```
C:\MyCerts>openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 1024 -out rootCA.crt
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:PK
State or Province Name (full name) [Some-State]:Punjab
Locality Name (eg, city) []:Multan
Organization Name (eg, company) [Internet Widgits Pty Ltd]:MyLab-CND
Organizational Unit Name (eg, section) []:IT
Common Name (e.g. server FQDN or YOUR name) []:RootCA
Email Address []:
```

Just press Enter on the Email Address field (leave it blank)

That will complete the Root CA generation. You should then see the command prompt return, and two files will exist in C:\MyCerts:

- rootCA.key (private key)
- rootCA.crt (the certificate)

Next: Step 4 — Generate the Server Key + CSR

Once Root CA is done, run:

```
C:\MyCerts>openssl genrsa -out server.key 2048

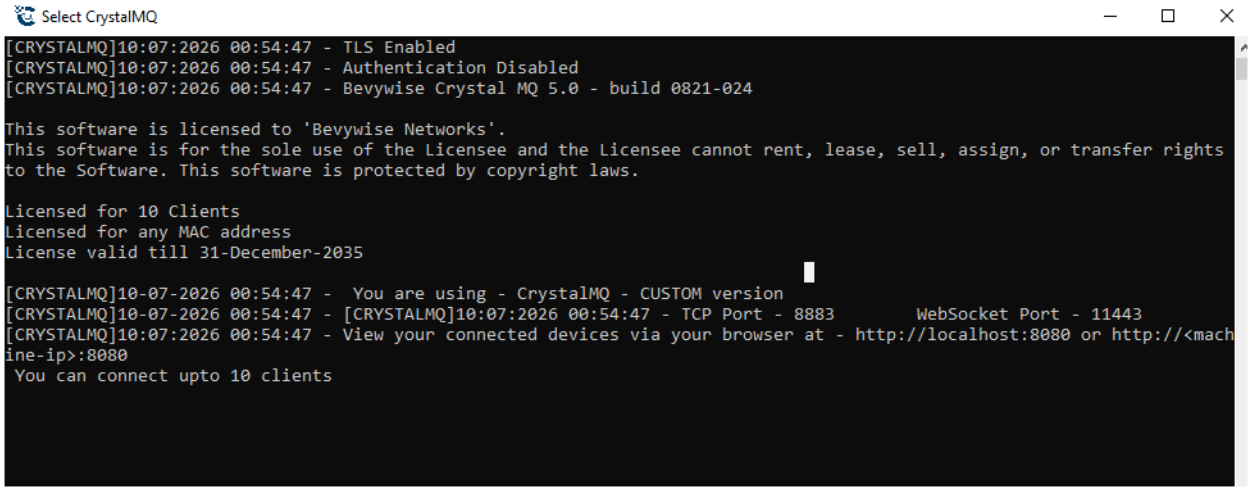
C:\MyCerts>openssl req -new -key server.key -out server.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:PK
State or Province Name (full name) [Some-State]:Punjab
Locality Name (eg, city) []:Multan
Organization Name (eg, company) [Internet Widgits Pty Ltd]:MyLab-CND
Organizational Unit Name (eg, section) []:IT
Common Name (e.g. server FQDN or YOUR name) []:192.168.100.138
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:Admin123
An optional company name []:GlobalTech
```

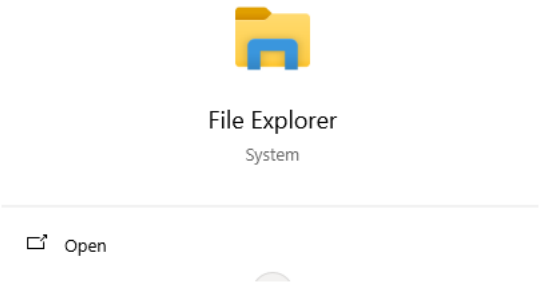
Step 5 — Sign the Server Cert with your Root CA:

```
C:\MyCerts>openssl x509 -req -in server.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial -out server.crt -days 500 -sha25
6
Certificate request self-signature ok
subject=C=PK, ST=Punjab, L=Multan, O=MyLab-CND, OU=IT, CN=192.168.100.138
```

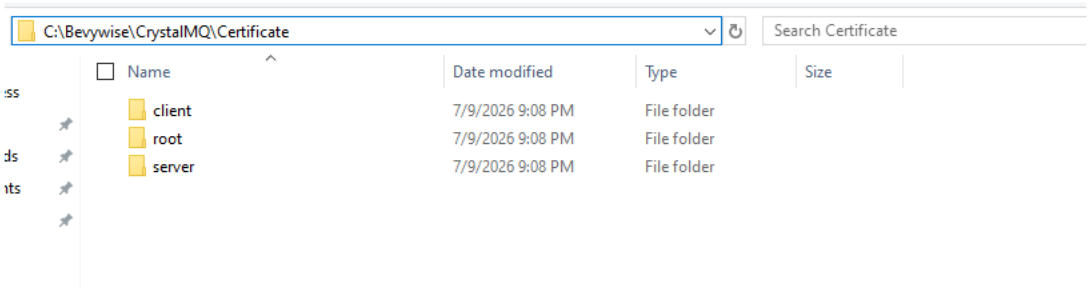
Now navigate to the Broker side (Web Server) and open the running cmd of CrystalMQ and closing it by pressing Ctrl + C twice



Open file explorer



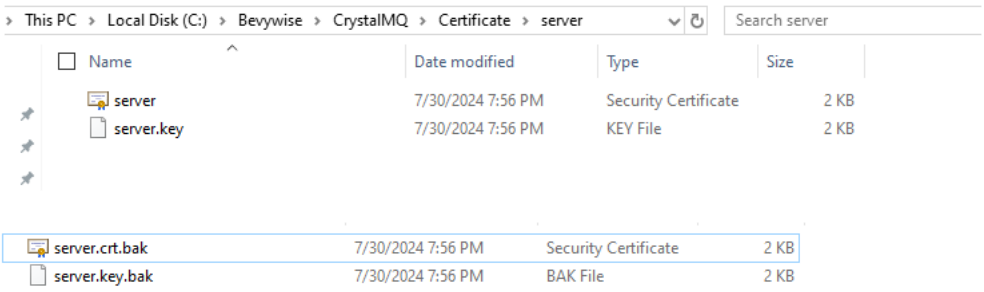
Navigate to the certificates area



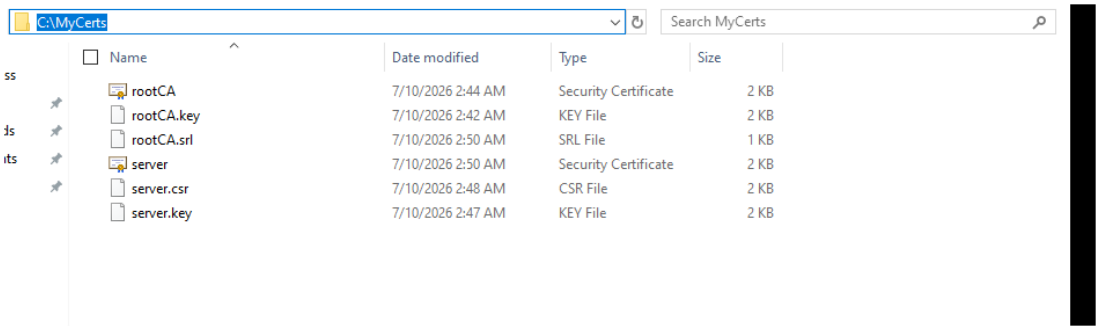
Select server and rename

server.key -> server.key.bak

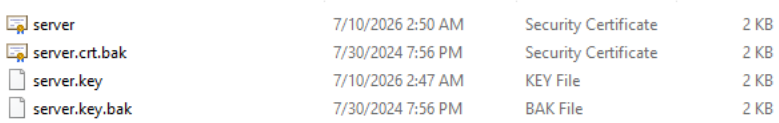
server -> server.crt.bak



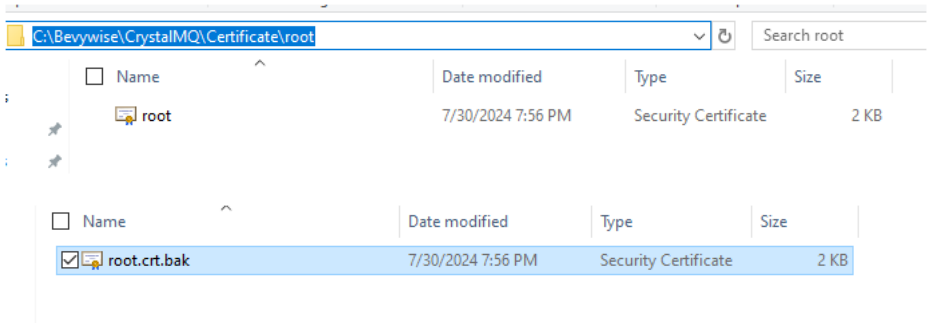
Now move the MyCerts folder where we made custom certificates



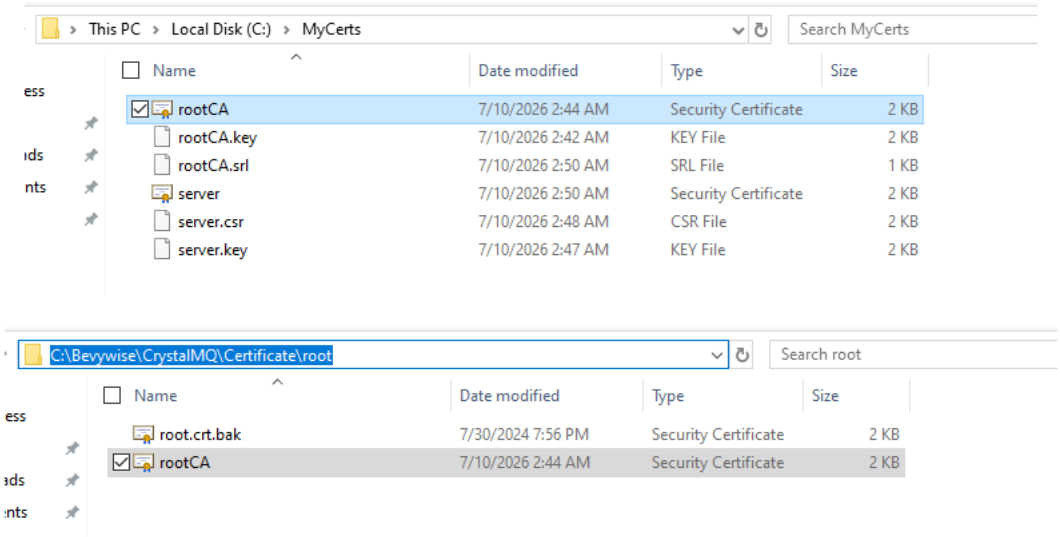
Now from here server.crt and server.key
copy these two and put them in the folder where we did changes



Now move to the root folder and rename the root -> root.crt.bak

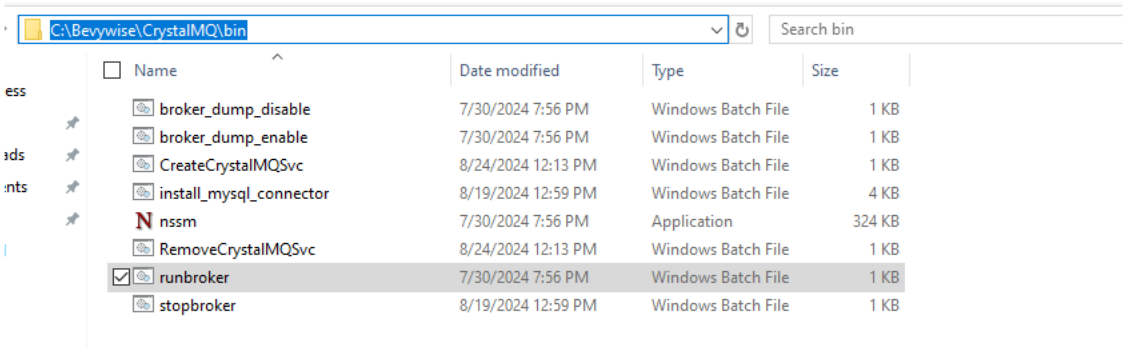


Now from custom made certificates in MyCerts copy the rootCA and put here



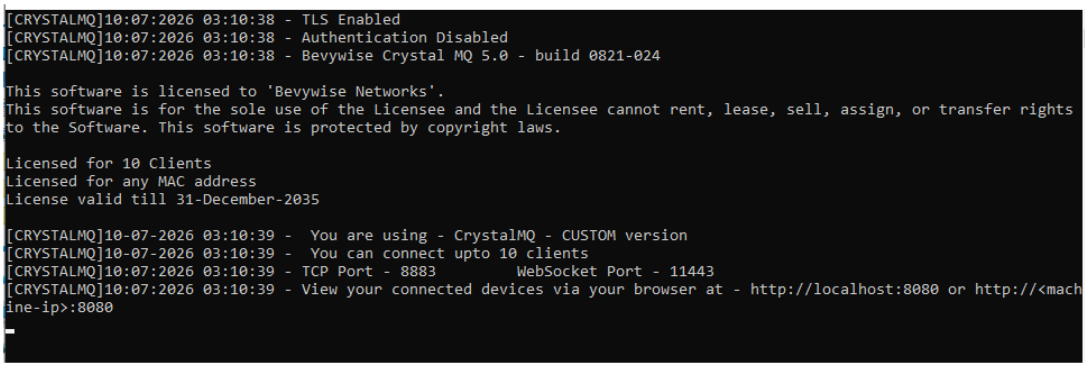
Now check wether the certificates are running perfectly or not

Run runbroker



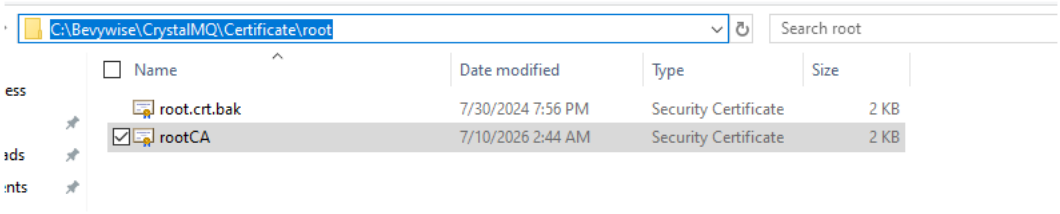
If this fails means, there is an error

But we didn't fail it means everything is working perfectly



Now let's move to the client (Admin Machine-1).

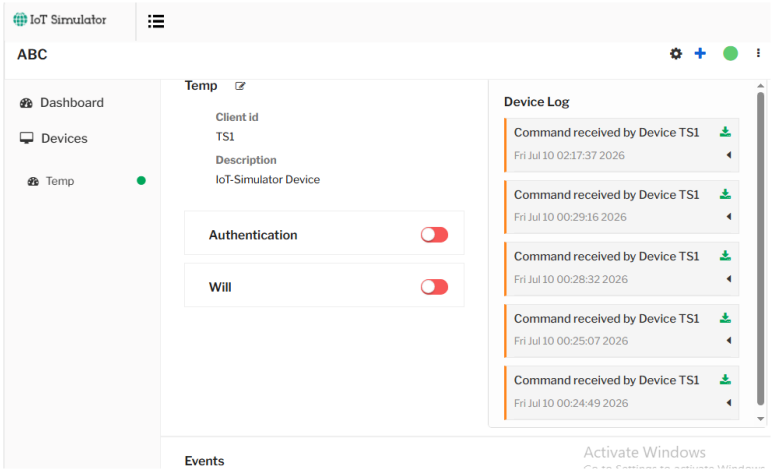
Now share/copy the rootCA certificate and store it in Admin VM (iot simulator)



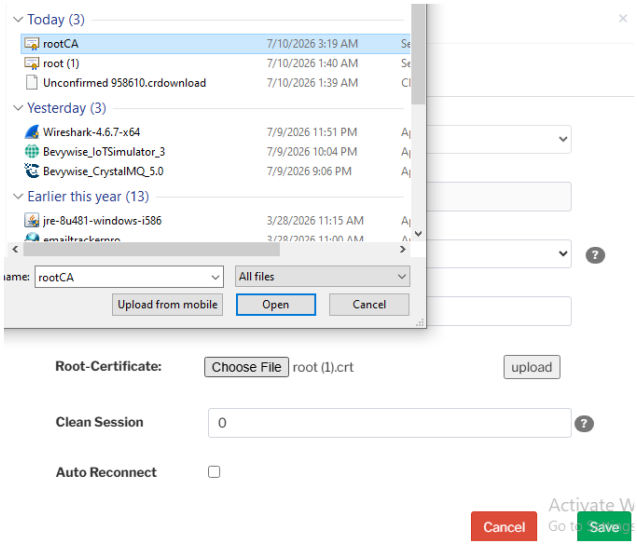
Successfully got the certificate in my Admin VM



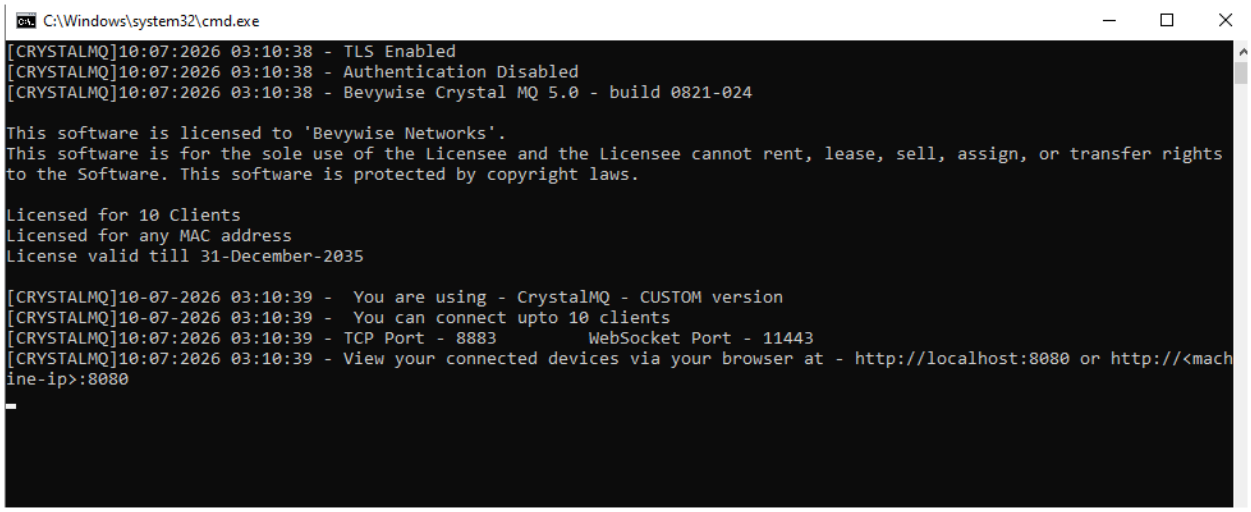
Now change the certificate and add the new rootCA custom made certificate here



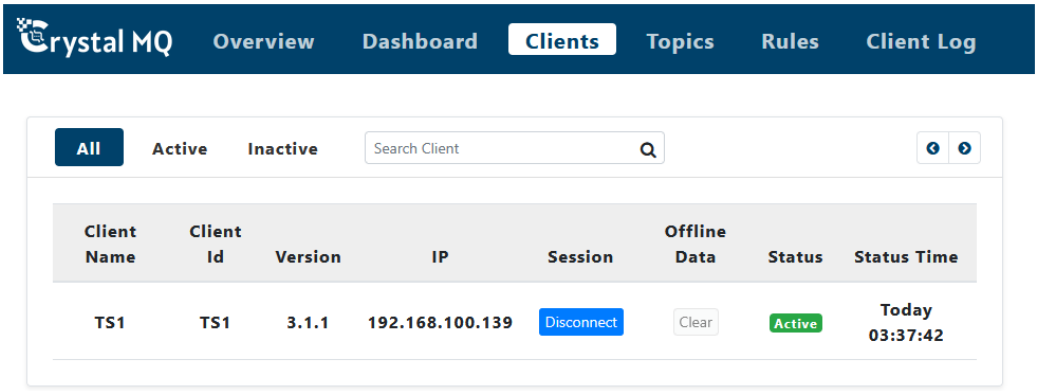
Click upload and save



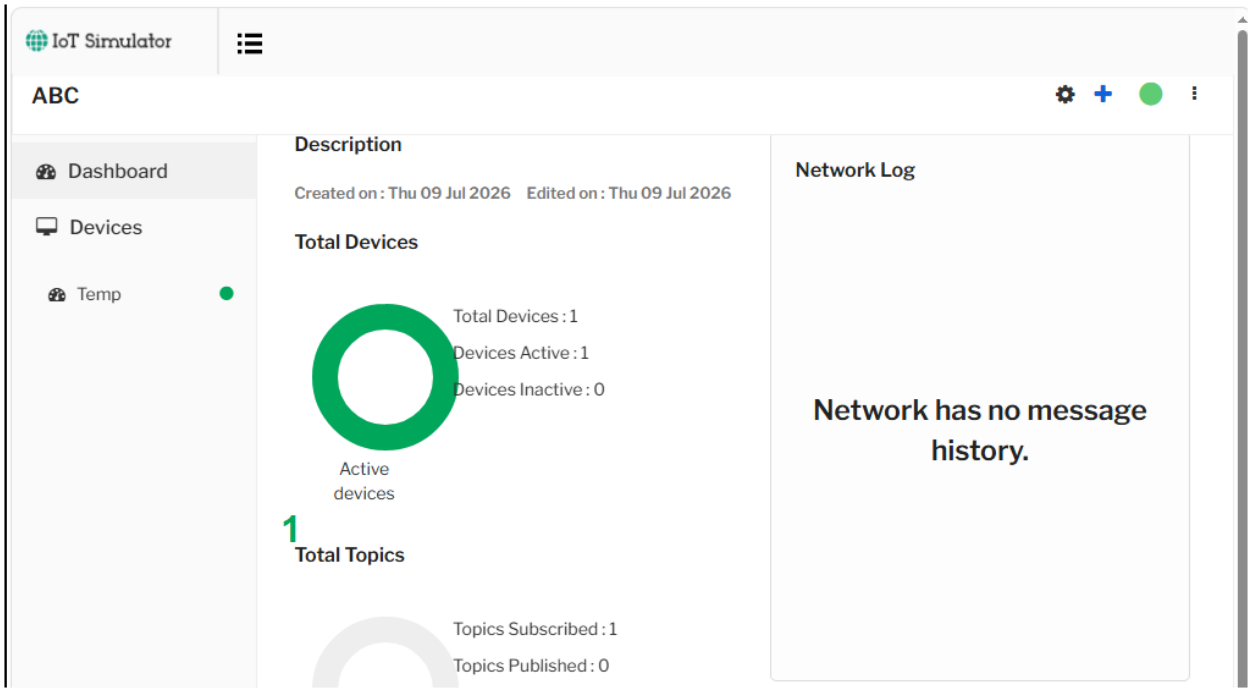
Close CRYSTAL MQ then rerun it again



Open Dashboard of Crystal MQ -> http://localhost:8080

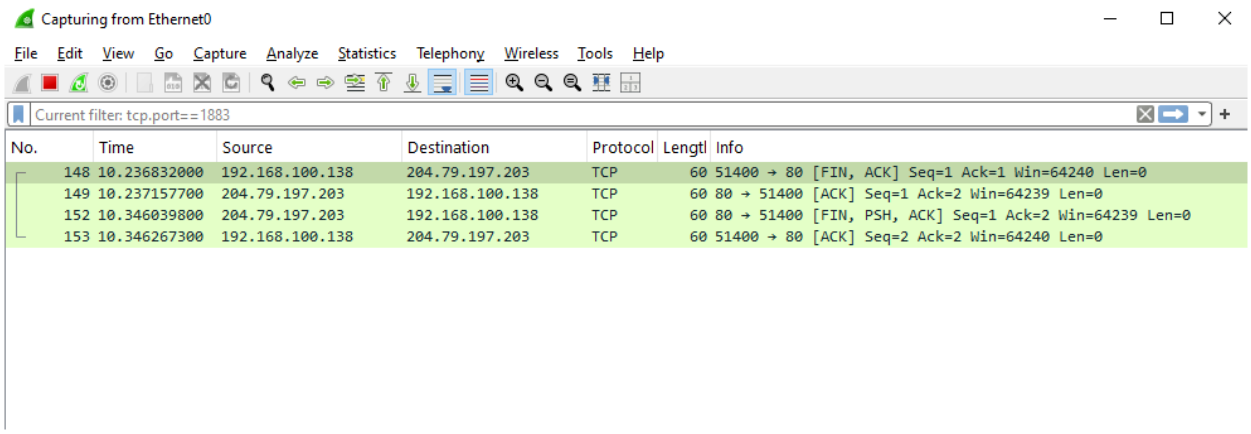


Now click on the green button to stop the network and then it will become red then click it again to make it green so that it will show that our self-signed certificate worked successfully

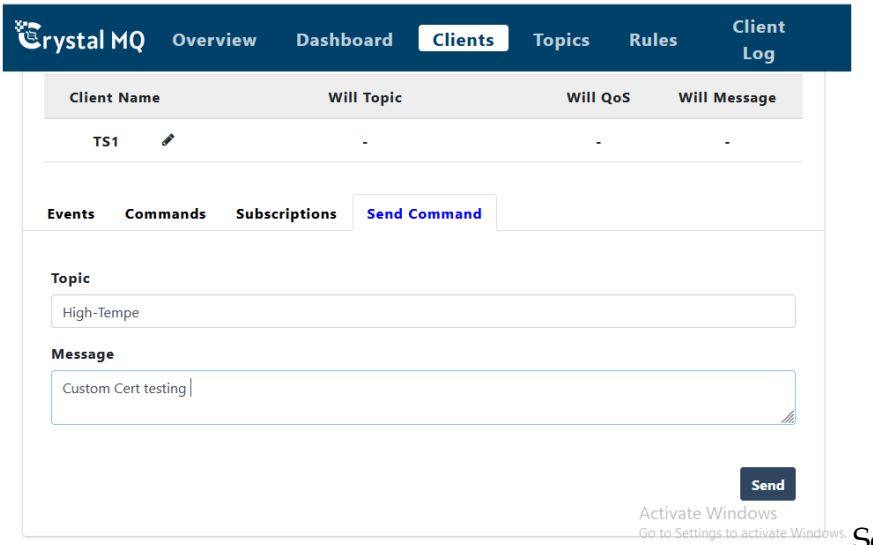


Now let's test that our data remains confidential or not
open Wireshark again in the capture mode in Admin VM and make it remain there

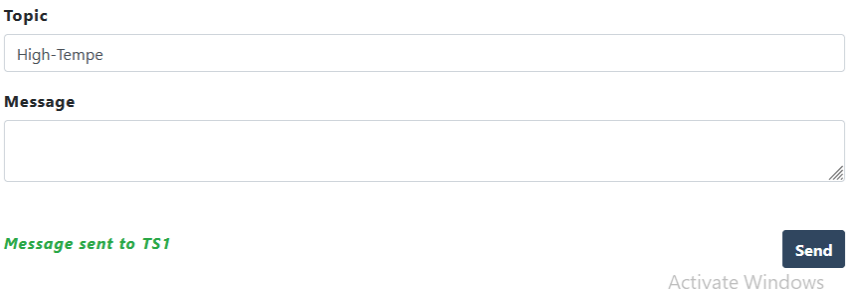
- Wireshark at capture state



Sending commands from Crystal MQ (Server/broker side)



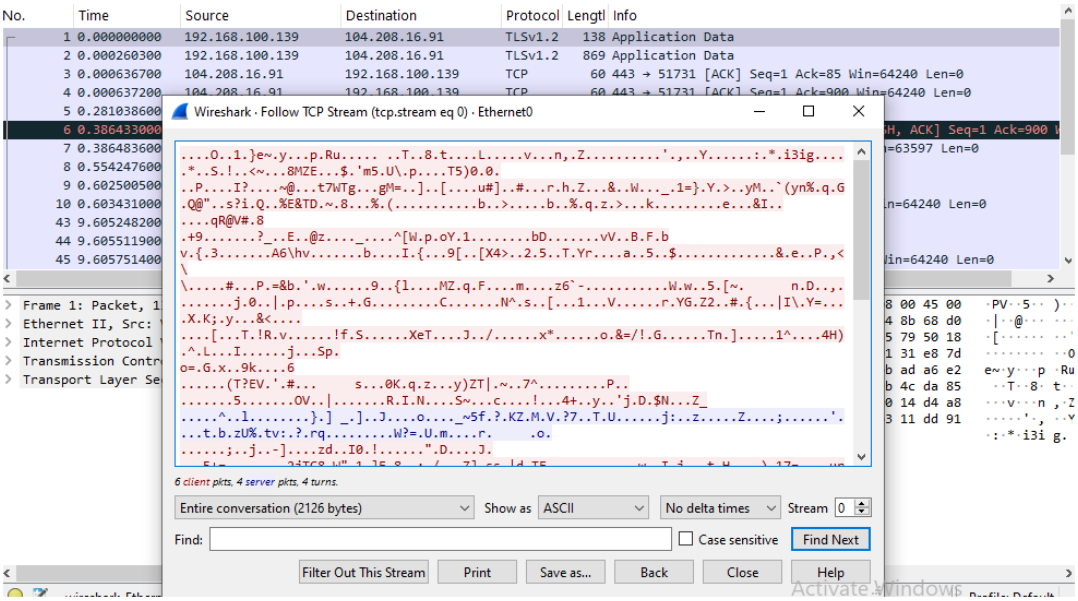
Sent



Open Wireshark & check

Filter: tcp.port==8883

Right click and select-> Follow/TCP Stream



Conclusion

This module successfully demonstrated the complete setup and security enhancement of an IoT communication environment using an MQTT-based architecture. The implementation involved configuring a Web Server as an MQTT Broker and an Admin Machine as an IoT client simulator, allowing virtual devices to communicate through the broker infrastructure.

During the initial testing phase, MQTT communication was analyzed without encryption. Using Wireshark packet capture and TCP stream analysis, it was verified that MQTT messages were transmitted in plaintext, exposing IoT data to potential interception by attackers. This highlighted the security risks associated with unsecured IoT communication protocols.

To overcome these security limitations, TLS encryption was successfully configured on the MQTT Broker and IoT Simulator. After enabling TLS and installing the required certificates, MQTT communication was transferred through a secure encrypted channel. Wireshark analysis confirmed that the transmitted data was no longer readable, proving that TLS successfully protected the confidentiality and integrity of IoT messages.

Additionally, the generation and implementation of custom self-signed certificates using OpenSSL provided practical understanding of PKI concepts, including Root Certificate Authority management, server certificate generation, and certificate-based authentication.

The successful completion of this module demonstrates the importance of implementing security controls within IoT environments. By combining proper network configuration, MQTT communication management, traffic analysis, and TLS encryption, a more secure and reliable IoT communication framework was achieved. This lab provided valuable practical knowledge of securing IoT infrastructures against unauthorized access and data interception.